

Practical Work 3: Document Databases and Hierarchical Data Models

Variation No. 3: The Fleet & Destination Planner

Ngoc Bao Tram Tran

231ADB294

Link to the video: <https://youtu.be/NUMum6Gh0ko>

Or <https://drive.google.com/file/d/1mNMvFz81yYCwb49-ikI7x1ex0IwHv5Sm/view?usp=sharing>

The Scenario: "AeroData Dynamics"

Welcome to **AeroData Dynamics**, a global leader in aviation logistics and fleet management. You have just been hired as a **Junior Data Engineer**.

The company is currently undergoing a massive digital transformation. Historically, flight data, airport registries, and fleet information have been stored in disjointed CSV and JSON files (legacy relational exports). This fragmentation is making it impossible for the "Fleet Strategy Team" to analyze aircraft efficiency or track global destination statistics in real-time.

Your Task: Your Chief Technology Officer (CTO) has tasked you with migrating this legacy data into a **MongoDB Document Database**. You must move the data from flat files into a hierarchical format, clean the data types, and perform a series of advanced analytical queries to determine future fleet investments.

Task 1: Installation process

1. A step-by-step description of how you installed MongoDB.

At the beginning, I downloaded MongoDB Community Server from <https://www.mongodb.com/try/download/community>. For MongoDB Community Server, I chose the current version 8.2.3, the Windows 64-bit platform, and the MSI package.

At the same time, I opened another page, <https://www.mongodb.com/try/download/compass>, to download MongoDB Compass. For MongoDB Compass, I chose the stable version 1.48.2, the Windows 64-bit (x64) platform, and the MSI package.

Then, I clicked the download button for MongoDB Community Server to install it. At first, the MongoDB 8.2.3 introduction page appeared. I clicked Next to continue with the setup wizard. After that, the End User License Agreement page appeared. I selected "I accept the terms in the License Agreement" and clicked Next. After that, I configured the component selection and service configuration settings. On the component selection screen, I also checked the option "Install MongoDB Compass." I then clicked Next until reaching the page "Ready to Install MongoDB 8.2.3 (64-bit)". I clicked Install and waited several minutes for the installation to complete.

For the mongosh terminal, I downloaded MongoDB Shell from <https://www.mongodb.com/try/download/shell>. The MongoDB Shell was provided as a ZIP file, so I extracted it into the MongoDB folder in Program Files (C:) - **C:\Program Files\MongoDB**. After that, I configured the system environment variables. In Edit the system environment variables, under System Variables, I added a new entry to Path that points to the bin directory of the extracted MongoDB Shell folder - **C:\Program Files\MongoDB\mongosh-2.5.10-win32-x64\bin**.

2. Screenshots of:

a. MongoDB Community Server Download page with your specific download Version, Platform and Package.

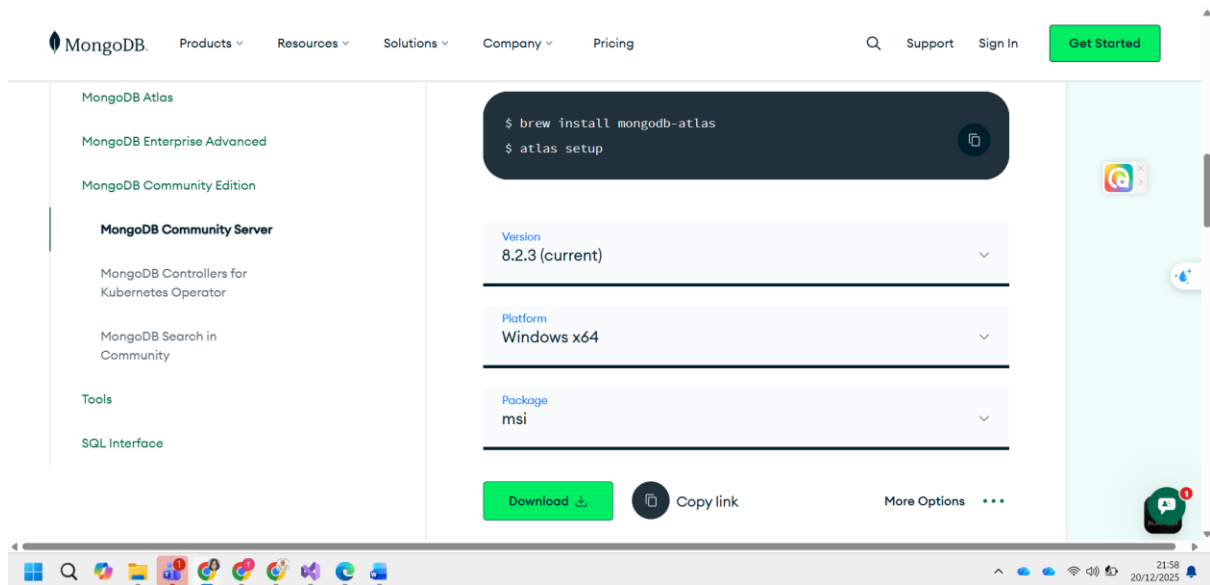


Figure 1. MongoDB Community Server Download page with Version: 8.2.3 (current), Platform: Windows x64 and Package: msi

b. MongoDB Compass Download page with your specific download Version, Platform and Package.

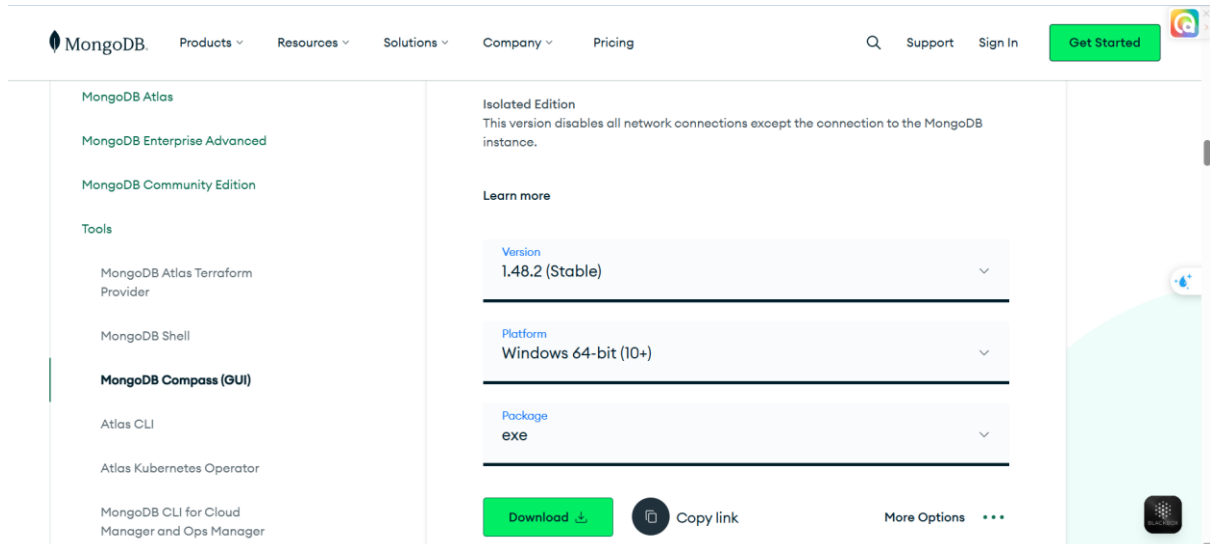


Figure 2. MongoDB Compass Download page with Version: 1.48.2 (Stable), Platform: Windows 64-bit (10+) and Package: exe

c. The "Component Selection" screen during installation.

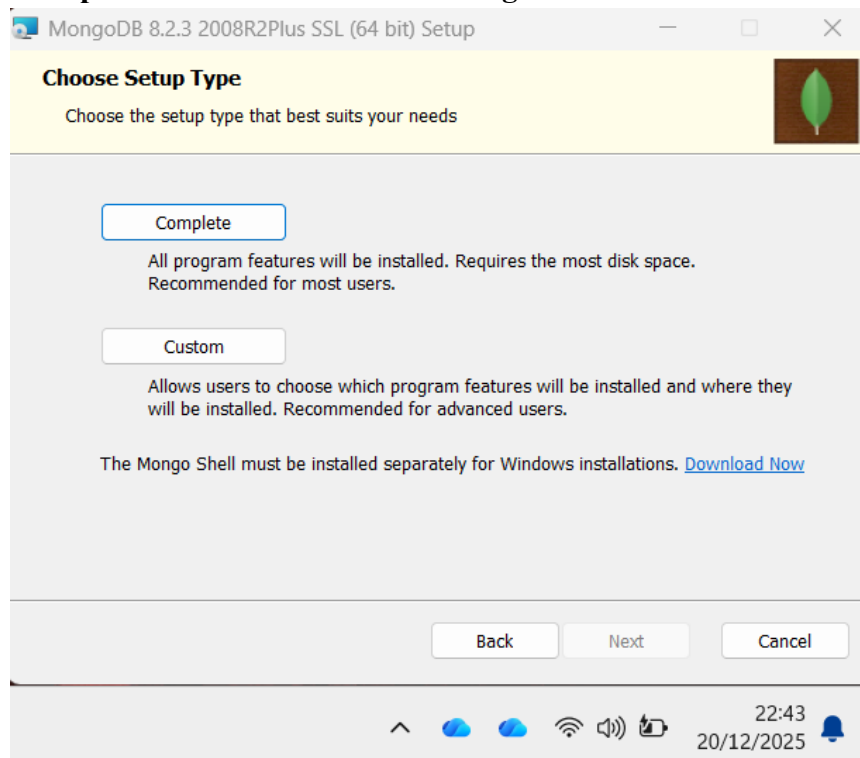


Figure 3. Component Selection screen showing the setup type options (Complete or Custom) during MongoDB installation – Selected Option is Complete

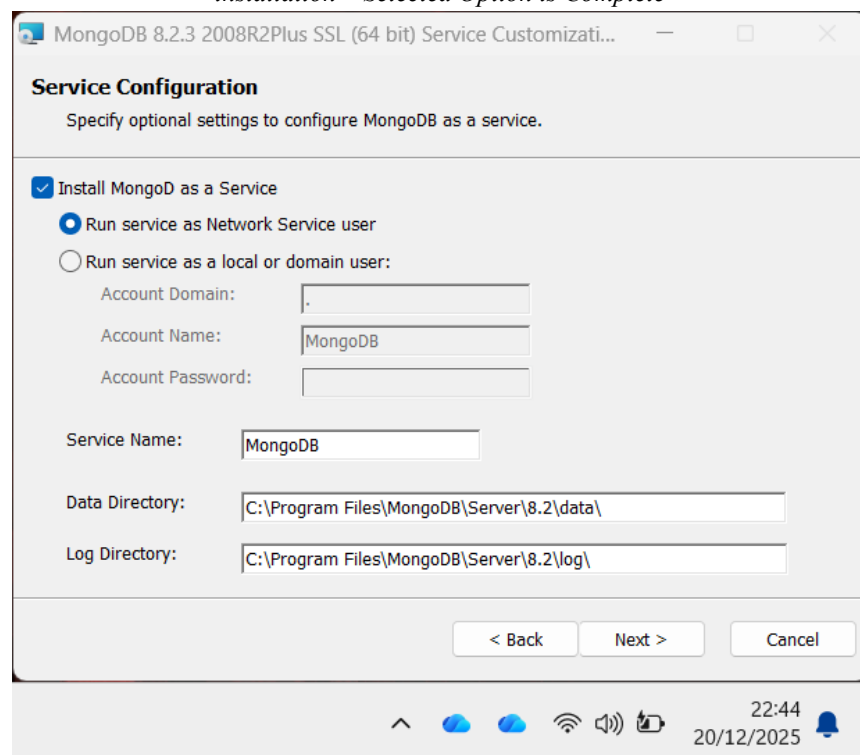


Figure 4. Service configuration screen for selecting MongoDB service components and runtime settings during installation | Selected Option is Install MongoDB as a Service - Run service as Network Service user

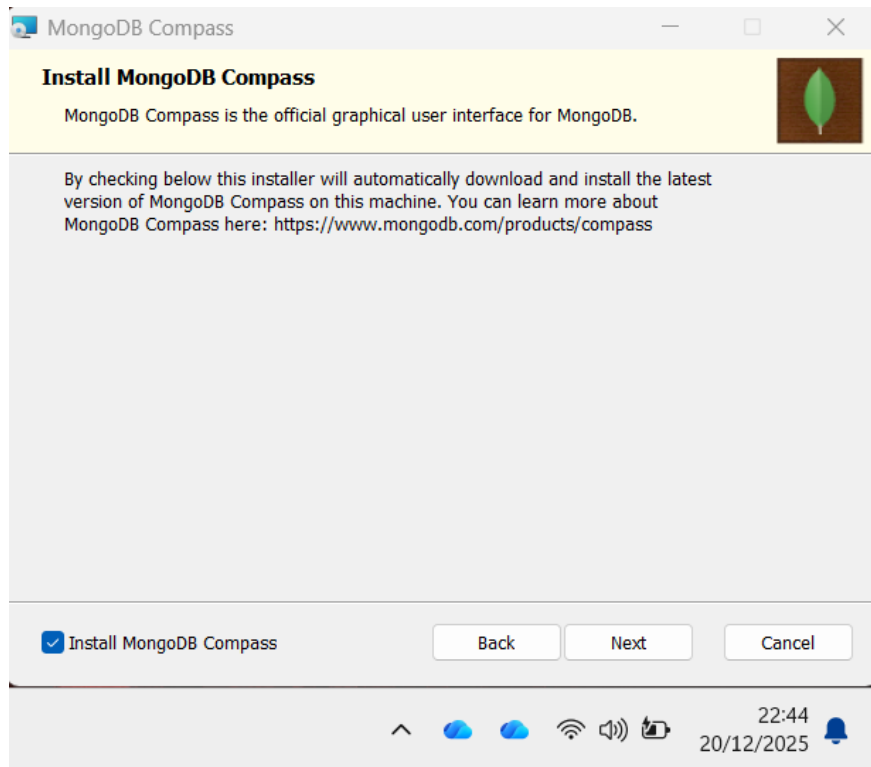


Figure 5. Optional component selection screen for installing MongoDB Compass during the MongoDB setup process.

d. The "Installation Complete" screen

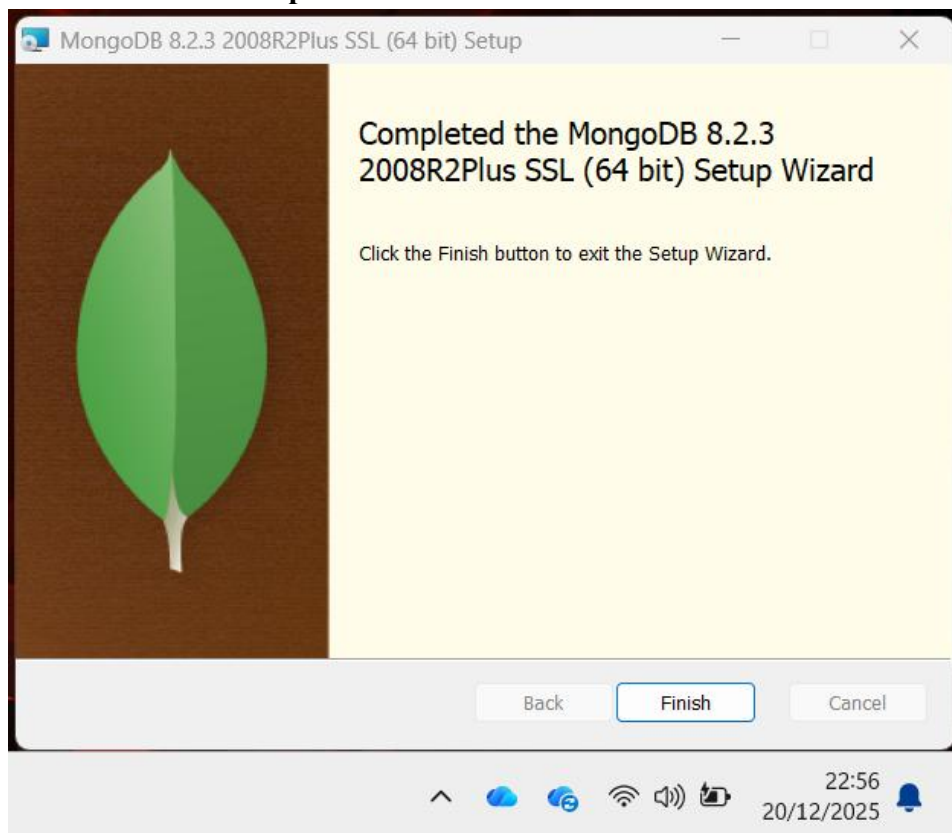
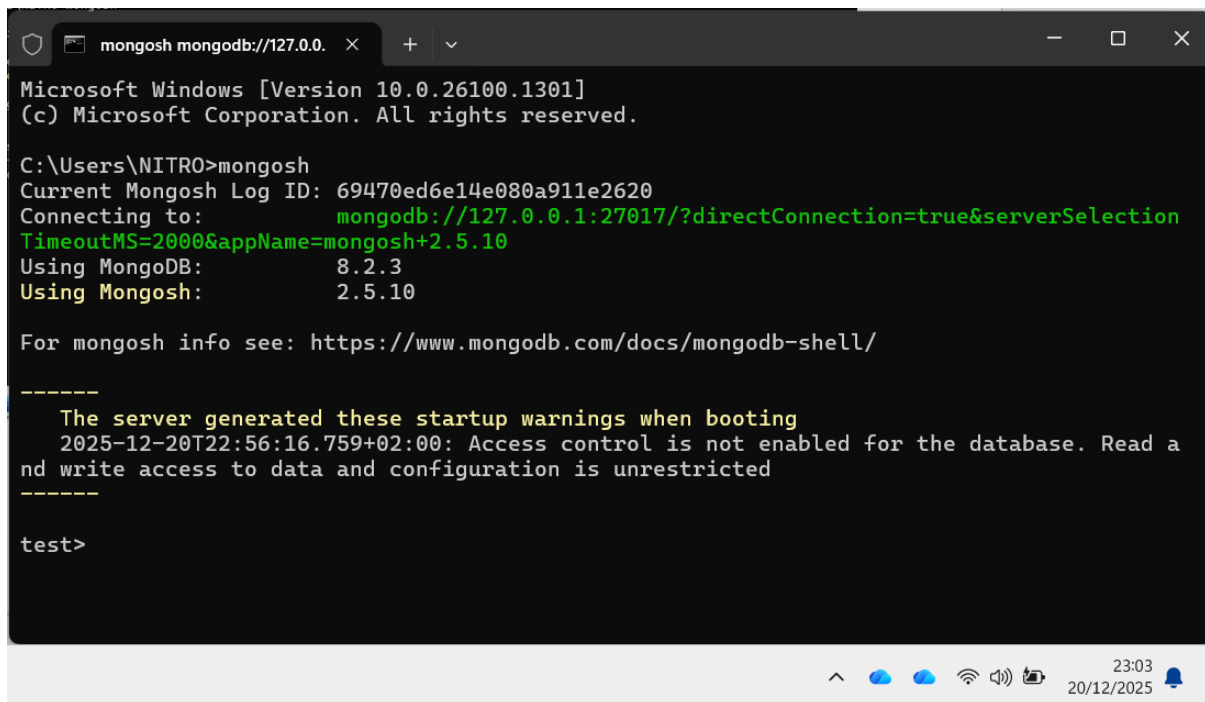


Figure 6. Completed the MongoDB 8.2.3 Setup Installation

e. A screenshot of the terminal showing a successful connection to mongosh



The screenshot shows a Windows terminal window with the title bar 'mongosh mongodb://127.0.0.1'. The terminal output is as follows:

```
Microsoft Windows [Version 10.0.26100.1301]
(c) Microsoft Corporation. All rights reserved.

C:\Users\NITRO>mongosh
Current Mongosh Log ID: 69470ed6e14e080a911e2620
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelection
TimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:  8.2.3
Using Mongosh:  2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-12-20T22:56:16.759+02:00: Access control is not enabled for the database. Read a
nd write access to data and configuration is unrestricted
-----

test>
```

The terminal window includes standard Windows taskbar elements at the bottom, such as system icons (network, volume, battery) and a clock showing 23:03 on 20/12/2025.

Figure 7. Terminal showing a successful connection to mongosh

Task 2: Data import and verification

1. **Description: Brief explanation of creating the database and the three collections**

First of all, I created a localhost connection (connected to localhost:27017). To do this, on the MongoDB Compass start page, I clicked New Connection, used the default URL `mongodb://localhost:27017`, and then clicked Connect. After connecting, the localhost instance contained three default databases: admin, config, and local.

Next, I clicked the plus sign (Create Database) under localhost:27017 to create a new database named aviationDB, and I declared the first collection as airlines.

Inside aviationDB, I created three collections: airlines (created at the same time as the database), and then airports and flight_logs, which were created using the Create Collection option within the aviationDB database.

The airlines collection, it is a simple JSON array of airlines and their hubs which stores airline information such as airline code (IATA), airline name (name), hub airport code (hub), and country. It acts as a lookup table for airline identity and hub information, which can be referenced when analyzing flight logs.

The airports collection, it is a list of known airports which stores airport details such as airport code (iata_code), airport name (name), city, country, and geographic coordinates - latitude (lat) and longitude – (lon).

The flight_logs collection, it is a raw log of recent flights which stores operational flight records, including flight identifier (flight_id), airline code (airline_iata), departure and arrival airport codes (origin_iata/ dest_iata), departure and arrival times (departure_time/ arrival_time), passenger_count, and aircraft_model.

The collections were imported from the corresponding provided .json and .csv files using the Import function in MongoDB Compass.

2. **Screenshots of:**

a. MongoDB Compass of the created database

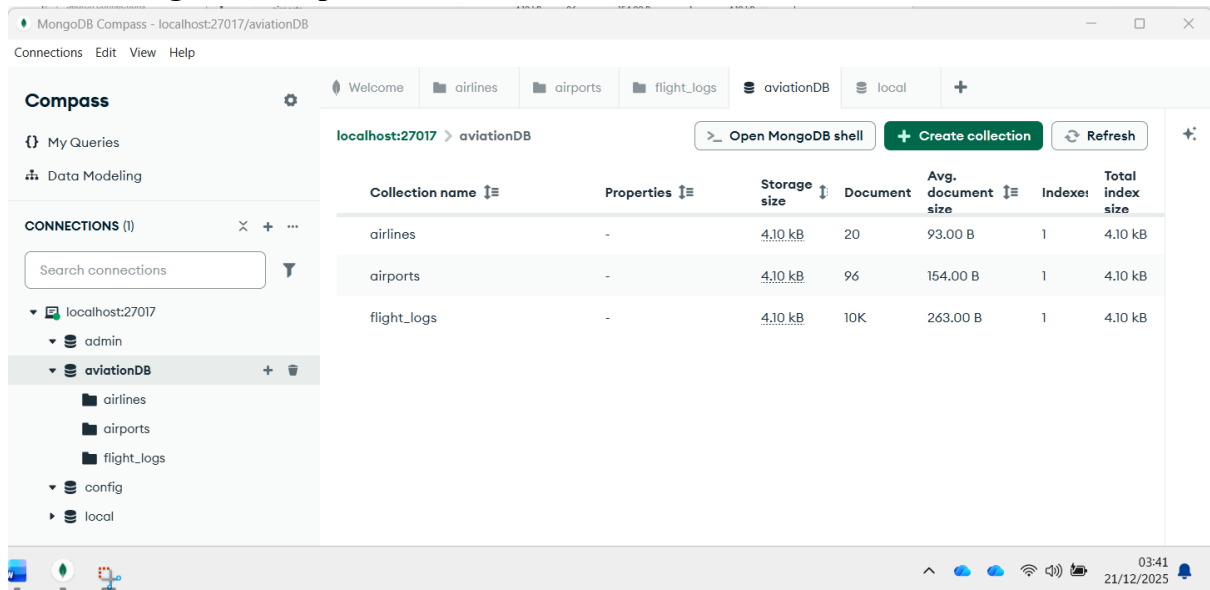


Figure 8. MongoDB Compass of the created database - aviationDB

b. MongoDB Compass for each of the 3 collections.

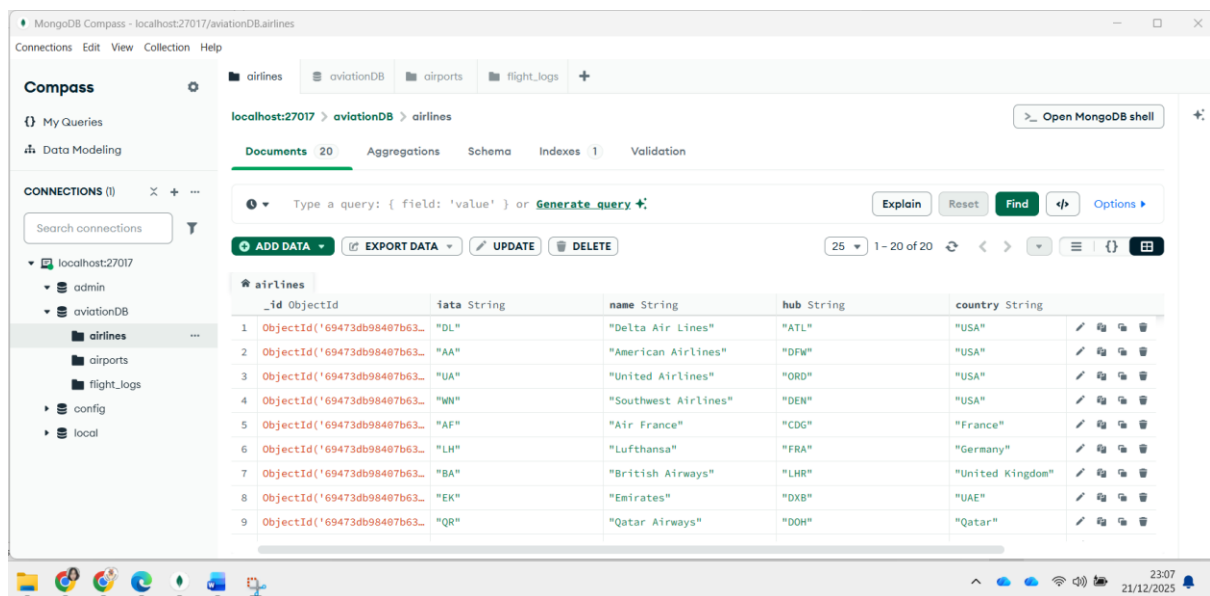


Figure 9. MongoDB Compass for collection "airlines"

Collection "airlines" - showing that all fields such as **iata**, **name**, **hub**, and **country** have data types set to **String** (except for the **_id** attribute, which is of type **ObjectId**).

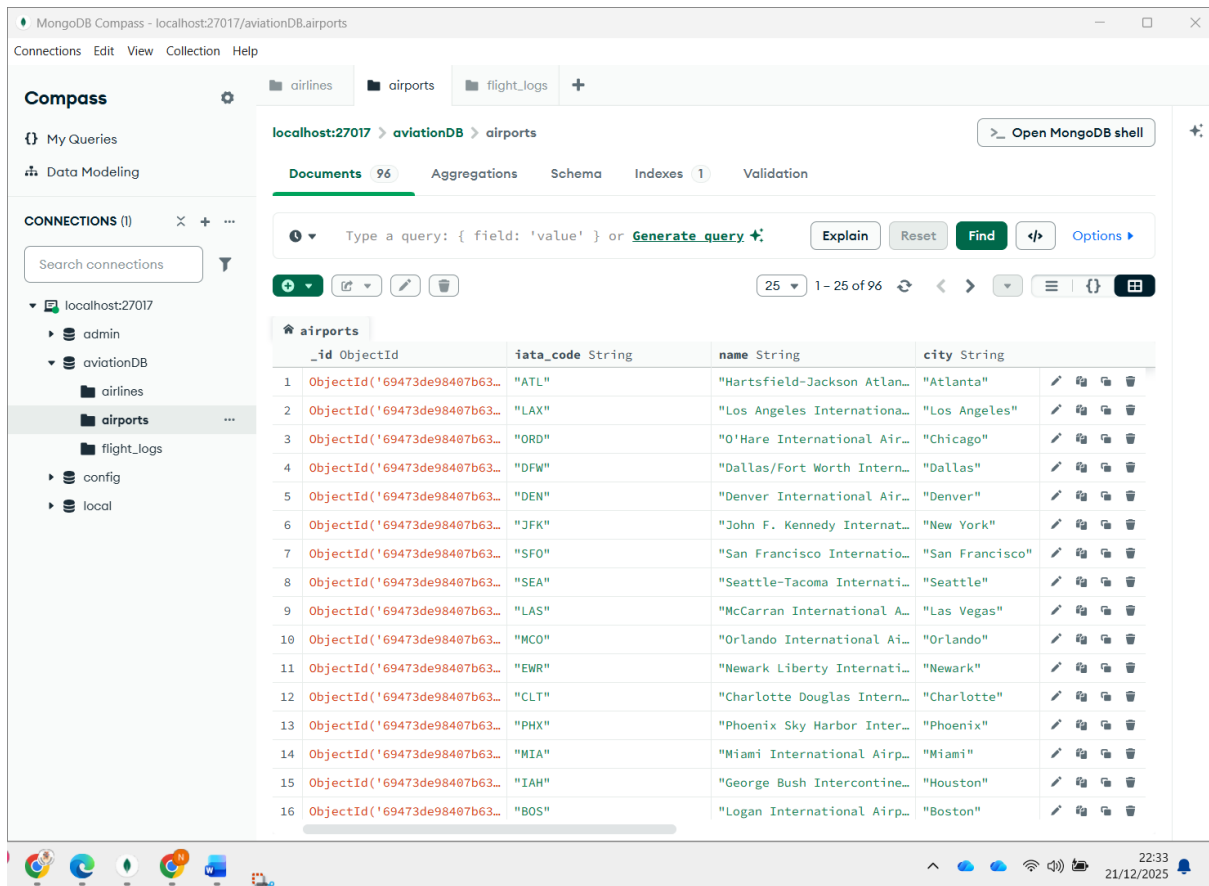


Figure 10.1. MongoDB Compass for collection “airports” [1]

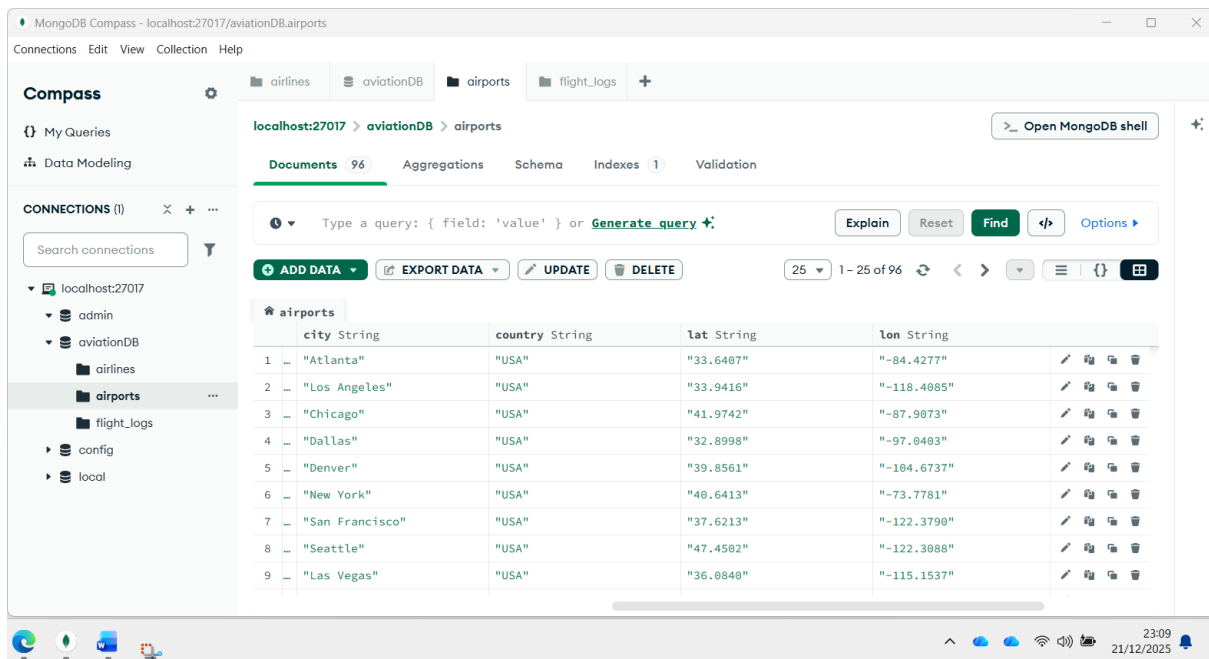


Figure 10.2. MongoDB Compass for collection “airports” [2]

Collection “airports” - showing that all fields such as `iata_code`, `name`, `city`, `country`, `lat`, and `lon` have data types set to **String** (except for the `_id` attribute, which is of type **ObjectId**).

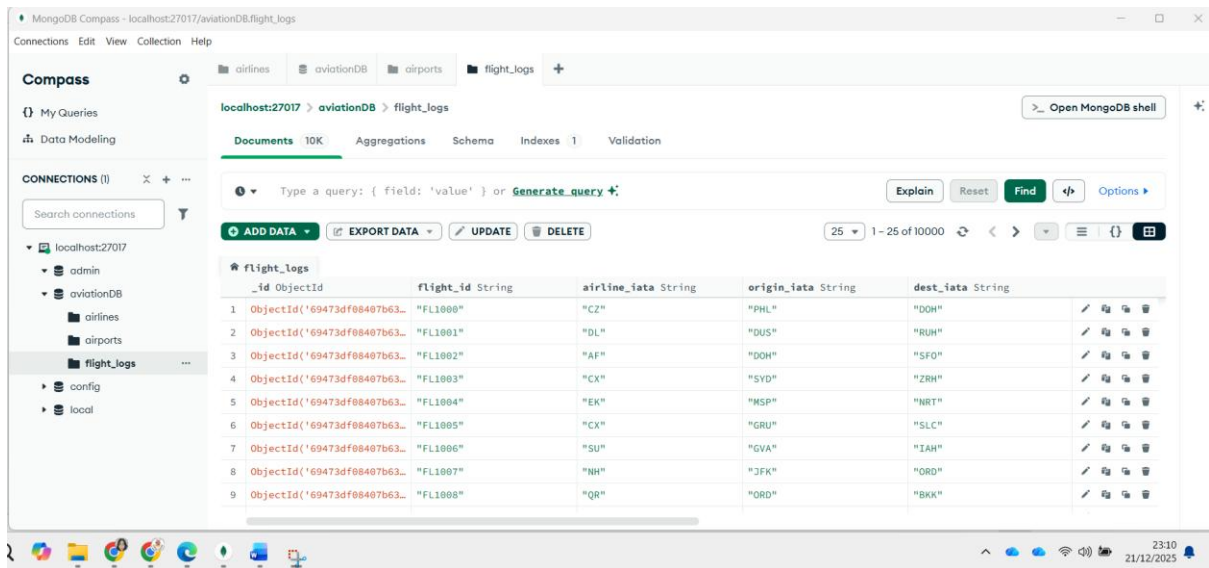


Figure 11.1. MongoDB Compass for collection “flight_logs” [1]

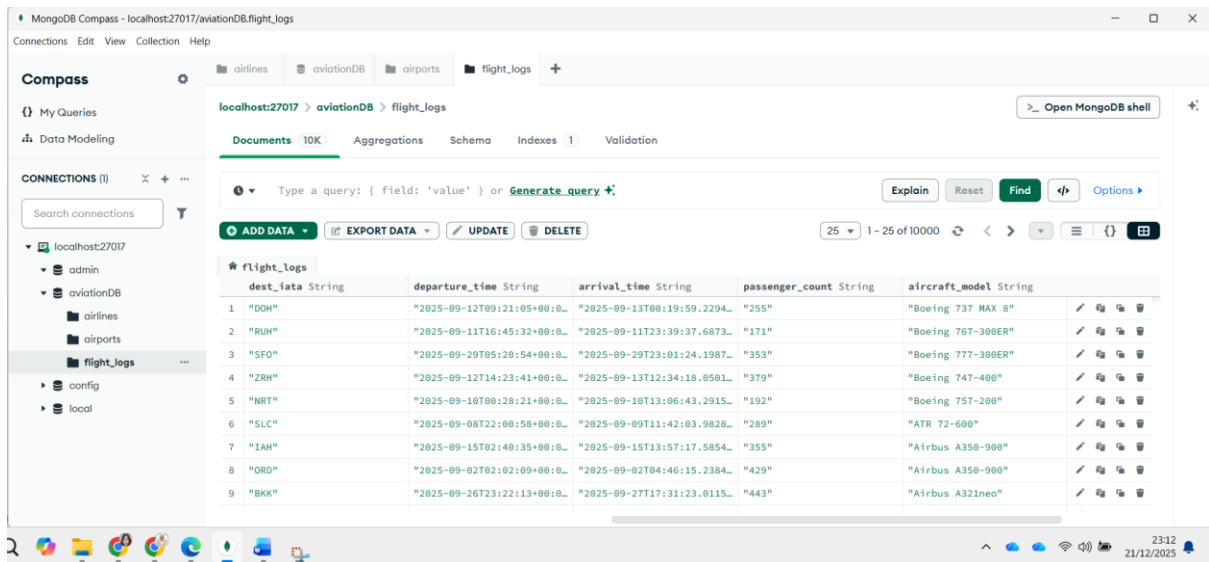


Figure 11.2. MongoDB Compass for collection “flight_logs” [2]

Collection “flight_logs” - showing that all fields such as **flight_id**, **airline_iata**, **origin_iata**, **dest_iata**, **departure_time**, **arrival_time**, **passenger_count**, and **aircraft_model** have data types set to **String** (except for the **_id** attribute, which is of type **ObjectId**).

Task 3: Data Type Correction

1. For collection “airlines”:

Because my variation states that for the **airlines** collection, I should **not change any data types** because the airlines collection already contains the correct data types. So, there is no screenshot for “the script”. The airlines collection still has all fields such as iata, name, hub, and country stored as String data types (except for the `_id` field, which is of type ObjectId).

Before + After implementation:

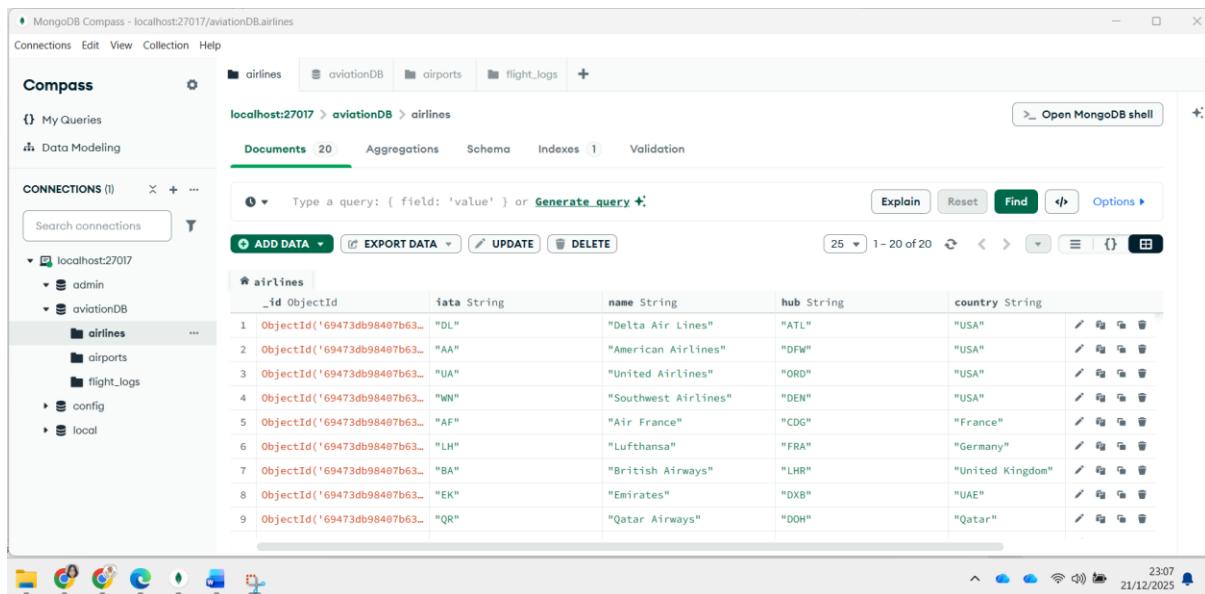


Figure 9. MongoDB Compass for collection “airlines”

2. For collection “airports”:

Task: Change the data types of: 1. “lat” from String to Double; 2. “lon” from String to Double

The Semantic Explanation (Purpose):

The purpose of converting the lat and lon fields from String to Double is to ensure that geographic coordinate data (lat is latitude and lon is longitude) is stored using appropriate numeric data types. Because the data type string attributes lat and lon cannot be analyzed well, as they limit the ability to perform numerical operations. In real life, latitude and longitude represent continuous numerical values and are commonly used in calculations such as distance measurement, geospatial analysis, and sorting. So, convert to data type double, and the data in these fields becomes semantically correct and suitable for analytical use.

Technical Explanation (Process):

The data type conversion was performed using the `updateMany()` method with an aggregation pipeline update in MongoDB. The update targets all documents in the airports collection with

the filter `{}`. The `$convert` operator was applied to transform the `lat` and `lon` fields from `String` to `Double`. The `onError` and `onNull` parameters were set to `null` to ensure that invalid or missing/null values do not cause the update operation to fail.

Before implementation:

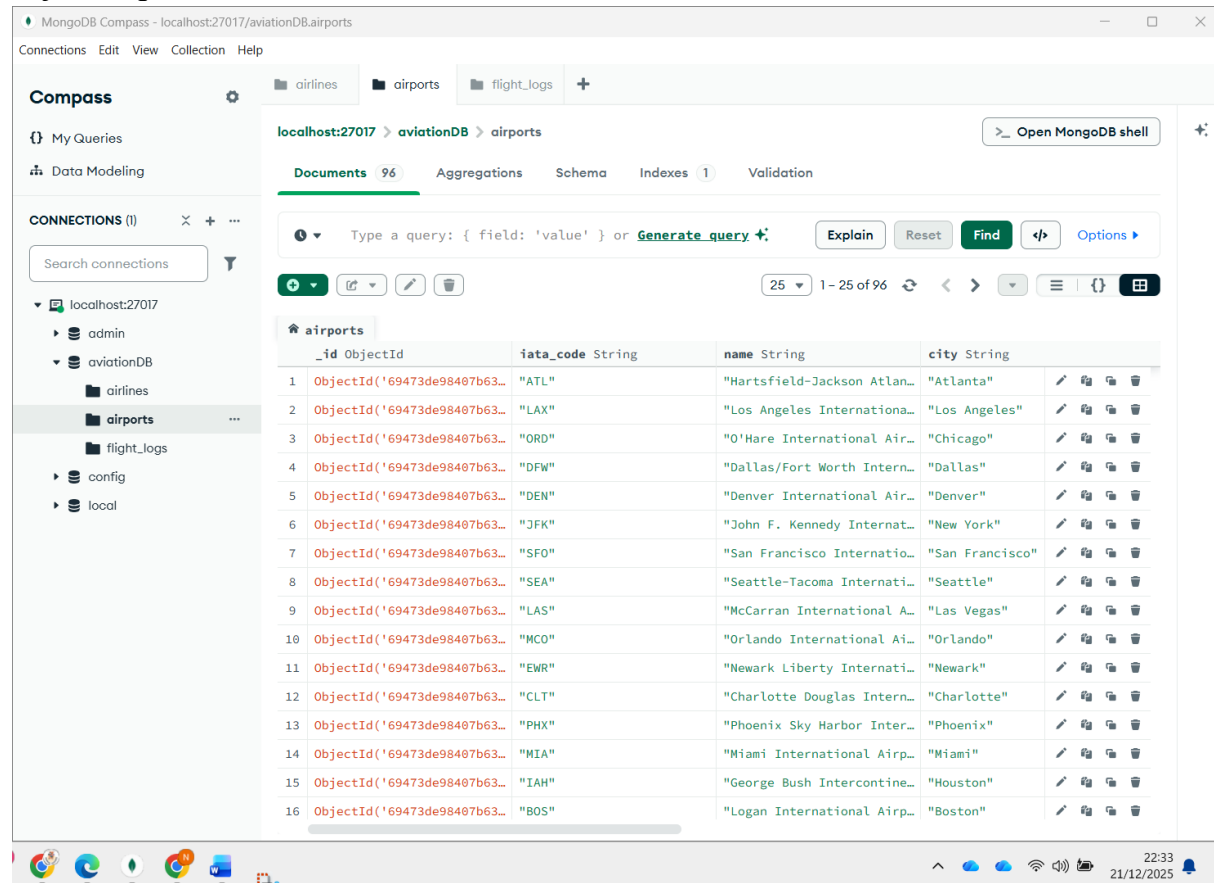


Figure 10.1. MongoDB Compass for collection “airports” before data type conversion [1]

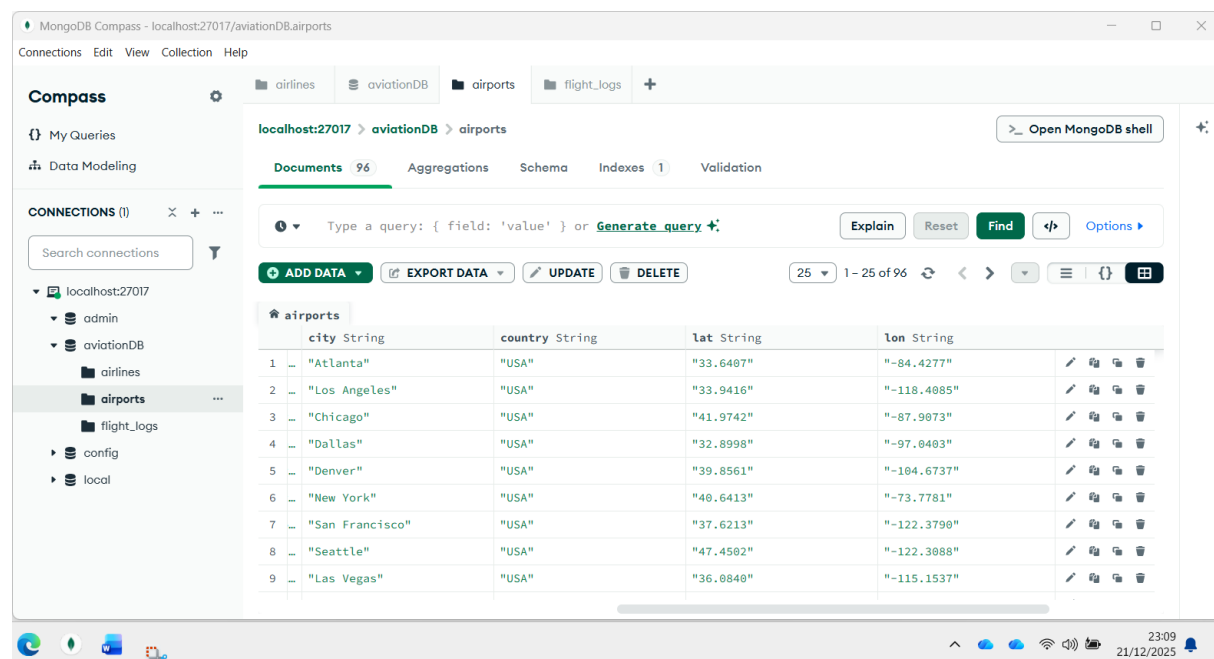
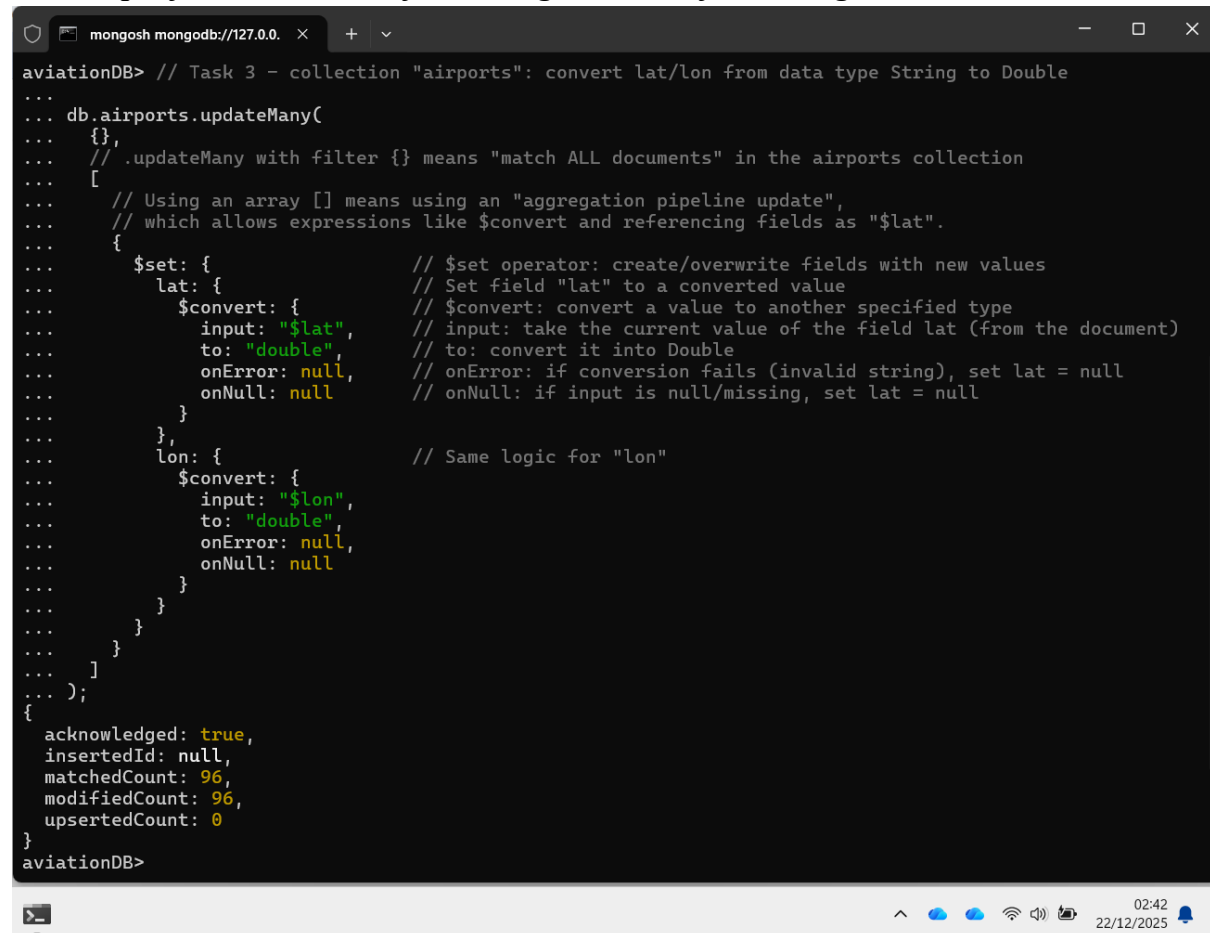


Figure 10.2. MongoDB Compass for collection “airports” before data type conversion [2]

Data types of lat and lon are string before data type conversion.

The Script of “lat” and “lon” from string to double; from string to double:



```
aviationDB> // Task 3 - collection "airports": convert lat/lon from data type String to Double
...
... db.airports.updateMany(
...   {},
...   // .updateMany with filter {} means "match ALL documents" in the airports collection
...   [
...     // Using an array [] means using an "aggregation pipeline update",
...     // which allows expressions like $convert and referencing fields as "$lat".
...     {
...       $set: {           // $set operator: create/overwrite fields with new values
...         lat: {           // Set field "lat" to a converted value
...           $convert: {     // $convert: convert a value to another specified type
...             input: "$lat", // input: take the current value of the field lat (from the document)
...             to: "double",  // to: convert it into Double
...             onError: null,  // onError: if conversion fails (invalid string), set lat = null
...             onNull: null   // onNull: if input is null/missing, set lat = null
...           }
...         },
...       },
...       lon: {             // Same logic for "lon"
...         $convert: {
...           input: "$lon",
...           to: "double",
...           onError: null,
...           onNull: null
...         }
...       }
...     ]
...   );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 96,
  modifiedCount: 96,
  upsertedCount: 0
}
aviationDB>
```

Figure 12. The script for changing the data types of lat and lon from string to double

Results after implementation:

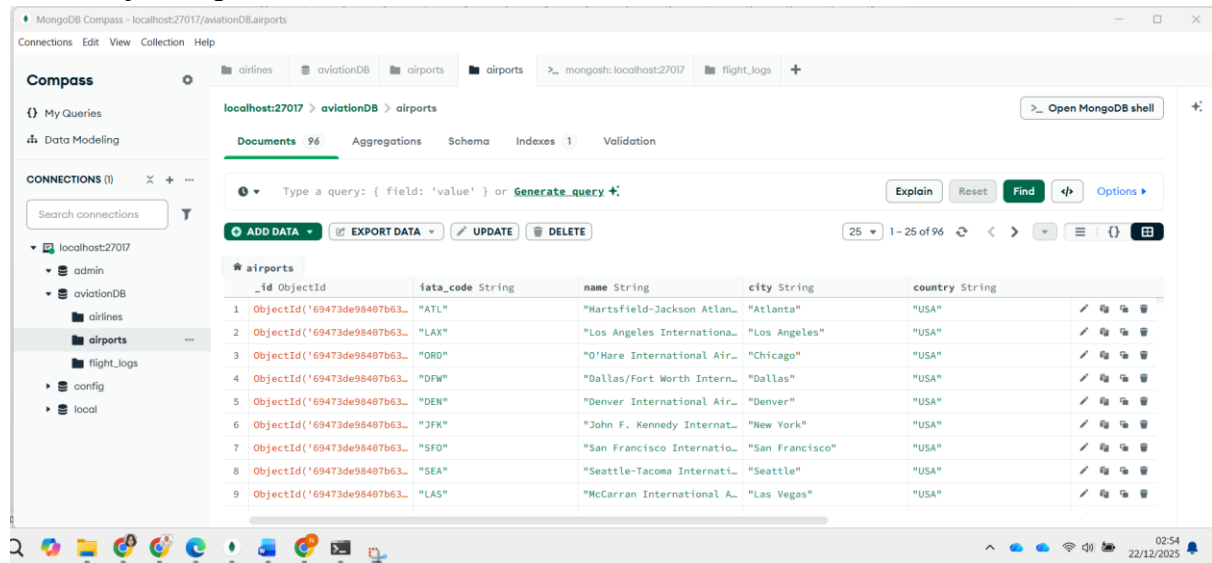


Figure 13.1. MongoDB Compass for collection “airports” after data type conversion

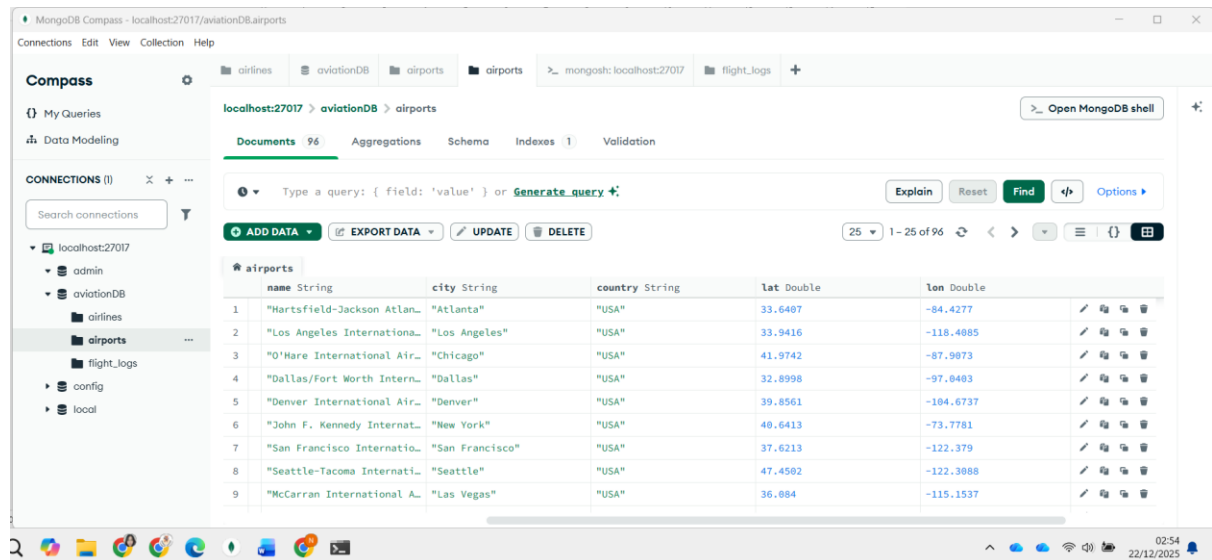


Figure 13.2. MongoDB Compass for collection “airports” after data type conversion

Data types of lat and lon are double after data type conversion.

3. For collection “flight_logs”:

3.1. Task: Change the data types of: 1. “departure_time” from String to Date; “arrival_time” from String to Date

The Semantic Explanation (Purpose):

The purpose of converting the departure_time and arrival_time fields from String to Date is to store the temporal information of these fields using an appropriate time-based data type. Storing the values of departure_time and arrival_time as strings limits time-related computations. These fields represent exact time and are important for time-related operations

such as calculating flight durations and sorting flights by departure or arrival time, etc. So, they are more suitable to be stored as the Date data type in order to fully utilize their potential.

Technical Explanation (Process):

The conversion was performed using the `updateMany()` method with an aggregation pipeline update on the `flight_logs` collection. The `$convert` operator was applied to transform the `departure_time` and `arrival_time` fields from string into Date type. The parameters `onError` and `onNull` were set to null to ensure that invalid or missing/null values do not interrupt the update process.

3.2. Task: Change the data type of: “passenger_count” from String to Int32

The Semantic Explanation (Purpose):

The purpose of converting the `passenger_count` field from String to Int32 is to ensure that data about the number of passengers is stored using an appropriate numeric data type. Because storing this value as a string limits numerical operations, it is not suitable for computation. With the integer data type (Int32), it allows the performance of operations such as aggregation, sorting, and statistical analysis, which the `passenger_count` field requires, because the `passenger_count` attribute represents a discrete count of passengers on each flight.

Technical Explanation (Process):

The conversion was carried out using the `updateMany()` method with an aggregation pipeline update on the `flight_logs` collection. The `$convert` operator was applied to transform the `passenger_count` field from the String data type to Int32 (integer). The parameters `onError` and `onNull` were set to null to safely handle invalid or missing values without interrupting the update process.

Before implementation:

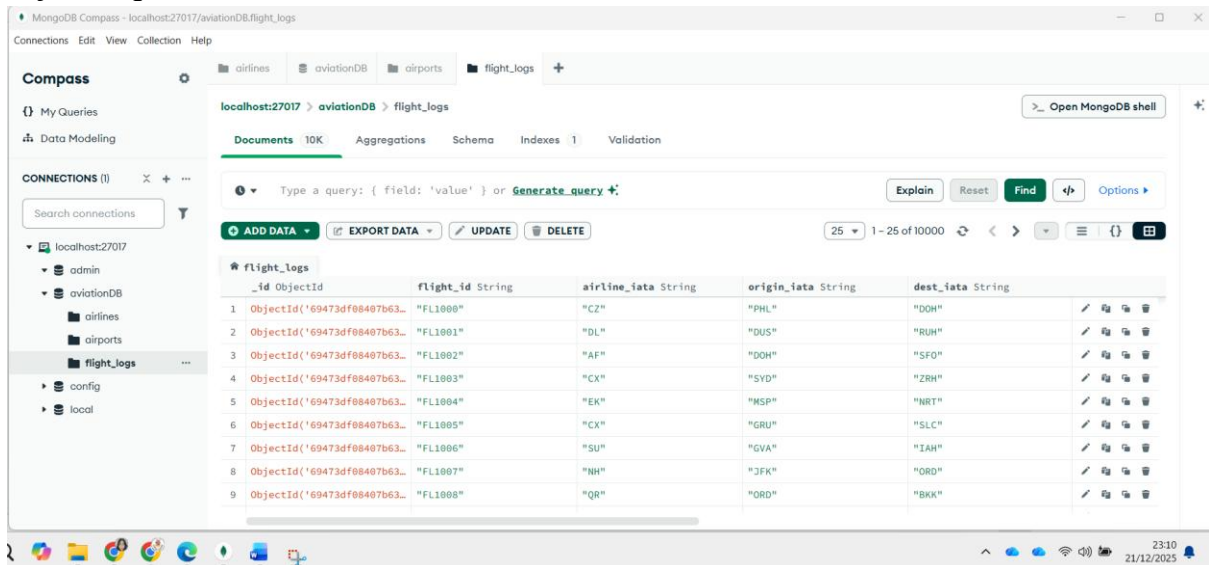


Figure 11.1. MongoDB Compass for collection “flight_logs” before data type conversion [1]

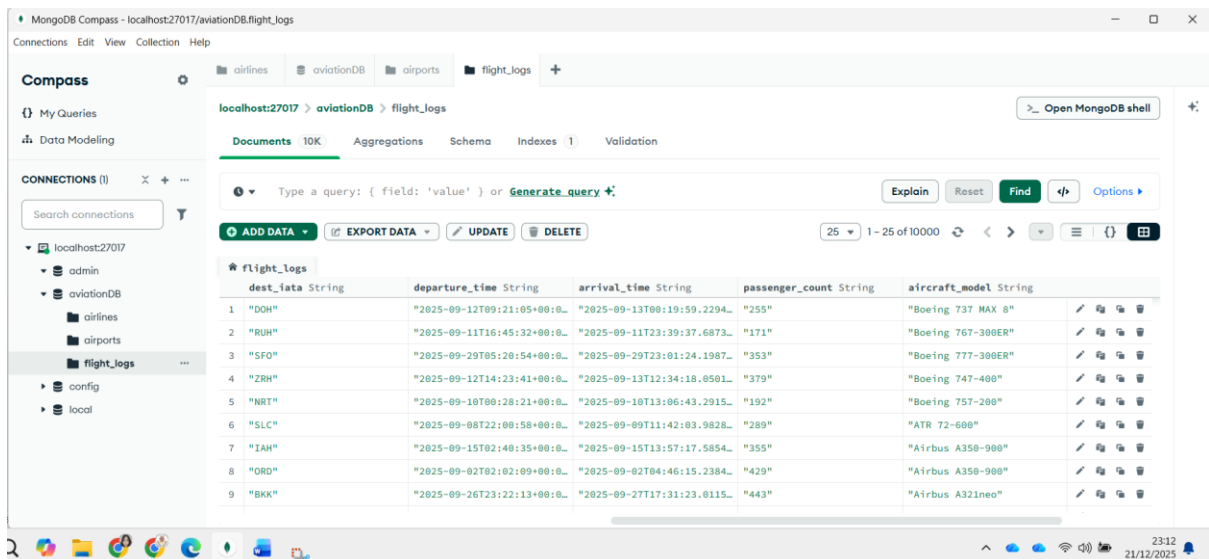


Figure 11.2. MongoDB Compass for collection “flight_logs” before data type conversion [2]

Data types of departure_time, arrival_time and passenger_count are string before data type conversion.

The Script of changing “departure_time” and “arrival_time” from string to date:

```
aviationDB> // Task 3 - collection "flight_logs":
... // convert departure_time and arrival_time from String to Date
...
... db.flight_logs.updateMany(
...   {},
...   // .updateMany with Filter: {} means "match ALL documents" in the flight_logs collection
...   [
...     // Update expressed as an aggregation pipeline (array syntax)
...     // which allows to use expressions like $convert and reference fields as "$departure_time".
...     {
...       $set: {
...         departure_time: {
...           $convert: {
...             input: "$departure_time", // Take the current value of departure_time from the document
...             to: "date", // Convert the string into Date type
...             onError: null, // If conversion fails (invalid string), set the value to null
...             onNull: null // If the input value is null or missing, set the value to null
...           }
...         },
...         arrival_time: { // Update the "arrival_time" field (same logic as above)
...           $convert: {
...             input: "$arrival_time",
...             to: "date",
...             onError: null,
...             onNull: null
...           }
...         }
...       }
...     ]
...   );
... {
...   acknowledged: true,
...   insertedId: null,
...   matchedCount: 10000,
...   modifiedCount: 10000,
...   upsertedCount: 0
... }
aviationDB>
```

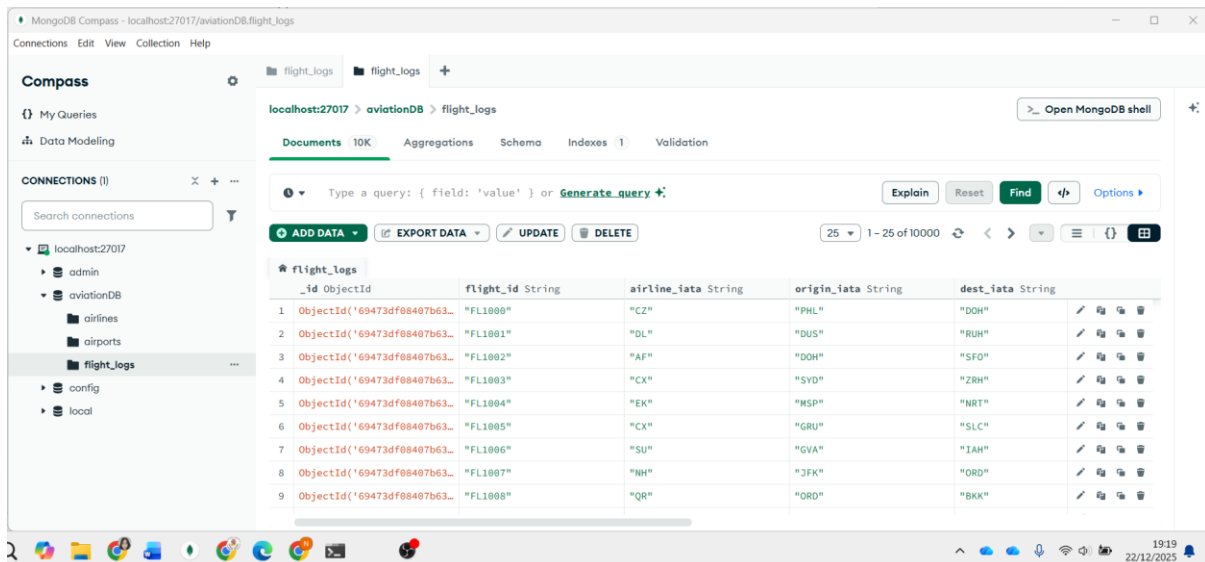
Figure 14. The script for changing the data types of departure_time and arrival_time from string to date

The Script of “passenger_count” from string to int32:

```
aviationDB> // Task 3 - collection flight_logs: Convert passenger_count from String to Int32
...
... db.flight_logs.updateMany(
...   {},
...   // .updateMany with Filter: {} means "match ALL documents" in the flight_logs collection
...   [
...     // Update defined as an aggregation pipeline (array syntax)
...     // which allows the use of expressions such as $convert and reference fields as "$passenger_count"
...     {
...       $set: {
...         passenger_count: {
...           $convert: {
...             // $set overwrites or creates the field with a new value
...             // Target field to be updated to a converted value
...             // $convert operator converts a value to another BSON data type
...             input: "$passenger_count", // Take the current passenger_count value from the document
...             to: "int", // Convert the string value to Int32
...             onError: null, // If conversion fails (invalid string), set value to null
...             onNull: null // If the input is null or missing, set value to null
...           }
...         }
...       }
...     }
...   ]
... );
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 10000,
  modifiedCount: 10000,
  upsertedCount: 0
}
aviationDB>
```

Figure 15. The script for changing the data types of passenger_count from string to int32

After implementation:



	_id	flight_id	airline_iata	origin_iata	dest_iata
1	ObjectId('69473df08407b63...')	1000	"CZ"	"PHL"	"DOH"
2	ObjectId('69473df08407b63...')	1001	"DL"	"DUS"	"RUH"
3	ObjectId('69473df08407b63...')	1002	"AF"	"DOH"	"SFO"
4	ObjectId('69473df08407b63...')	1003	"CX"	"SYD"	"ZRH"
5	ObjectId('69473df08407b63...')	1004	"EK"	"MSP"	"NRT"
6	ObjectId('69473df08407b63...')	1005	"CX"	"GRU"	"SLC"
7	ObjectId('69473df08407b63...')	1006	"SU"	"GVA"	"IAH"
8	ObjectId('69473df08407b63...')	1007	"NH"	"JFK"	"ORD"
9	ObjectId('69473df08407b63...')	1008	"QR"	"ORD"	"BKK"

Figure 16.1. MongoDB Compass for collection “flight_logs” after data type conversion [1]

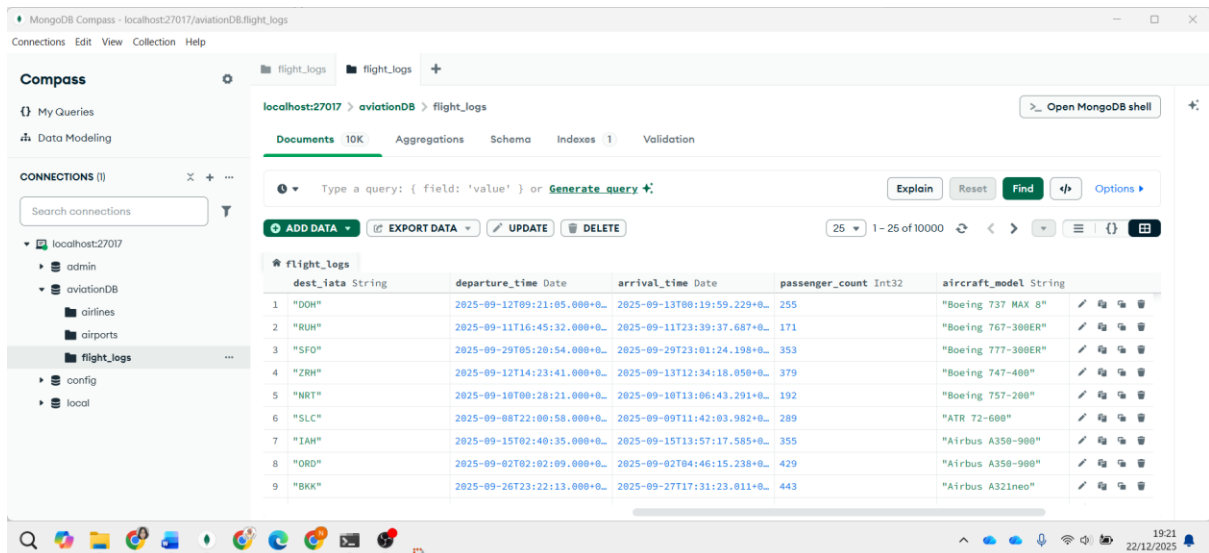


Figure 16.2. MongoDB Compass for collection “flight_logs” after data type conversion [2]

After data type conversion, data types of `departure_time`, `arrival_time` are date and data type of `passenger_count` is int32.

Task 4: Denormalization

1. Must provide:

a. Scripts title: “flights_enhanced”

b. Purpose (Semantic explanation):

The purpose of this script is to prepare flight data for easier analysis by combining flight records from "flight_logs" collection data with their destination airport details from "airports" collection and precomputing flight duration into a new collection called flights_enhanced (denormalized collection), so that analytical queries can be performed without additional joins or calculations.

c. Process (Technical explanation):

The pipeline starts on the flight_logs collection using an aggregation framework workflow.

- \$lookup left joins "flight_logs" collection data with the "airports" collection data by matching dest_iata (in flight_logs) to iata_code (in airports), and stores the matched airport document in field "dest_airport_details".
- \$unwind converts dest_airport_details from an array into a single embedded document, while preserveNullAndEmptyArrays: true keeps flight records even if no matching airport is found.
- \$addFields creates a new field flight_duration_minutes using \$dateDiff to compute the time difference between departure_time and arrival_time in minutes.
- \$out writes all transformed documents into a new collection named flights_enhanced, creating or overwriting it as needed.

2. Screenshots of:

Before the implementation:

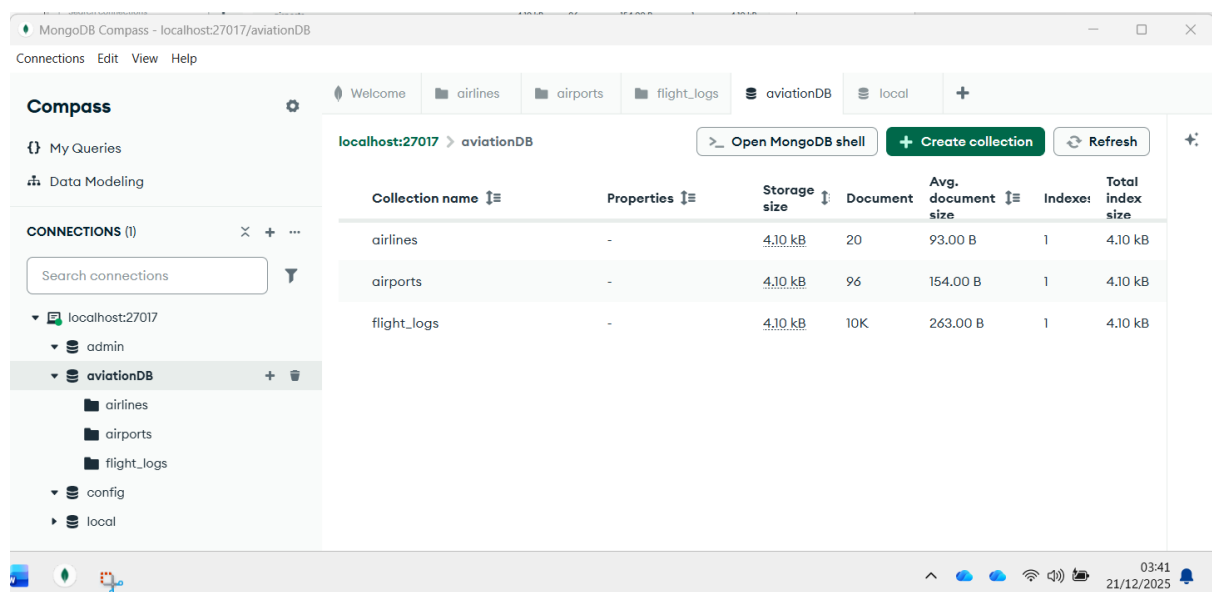


Figure 8. MongoDB Compass of the created database – aviationDB [Before with only 3 collections]

a. *The script itself*

```

>_MONGOSH
> db.flight_logs.aggregate([
  // Stage 1: Look up and embed related data
  {
    $lookup: {
      // Join flight_logs with the airports collection
      from: "airports",
      // Match flight_logs.dest_iata with airports.iata_code
      localField: "dest_iata",
      foreignField: "iata_code",
      // Embed the matched airport document into a new field called dest_airport_details
      as: "dest_airport_details"
    }
  },
  // Unwind from array to object
  {
    $unwind: {
      // $unwind with $dest_airport_details:
      // convert dest_airport_details from an array into a single embedded document
      path: "$dest_airport_details",
      preserveNullAndEmptyArrays: true
      // Keep the flight record even if no matching airport is found
    }
  },
  // Stage 2: Calculate a new field
  {
    $addFields: {
      flight_duration_minutes: {
        // Create a new field that calculates the flight duration in minutes
        $dateDiff: {

```

Figure 17. 1. MongoDB Compass Shell showing denormalization aggregation pipeline (Stage 1–2) [1]

```

        preserveNullAndEmptyArrays: true
        // Keep the flight record even if no matching airport is found
      }
    },
    // Stage 2: Calculate a new field
    {
      $addFields: {
        flight_duration_minutes: {
          // Create a new field that calculates the flight duration in minutes
          $dateDiff: {
            // $dateDiff returns the integer difference between the startDate and endDate
            // measured in the specified units
            // The duration is calculated by subtracting departure_time from arrival_time
            startDate: "$departure_time",
            endDate: "$arrival_time",
            unit: "minute"
          }
        }
      }
    },
    // Stage 3: Output
    {
      // Output all transformed documents into a new collection
      // Create (or overwrite) the collection named flights_enhanced
      $out: "flights_enhanced"
    }
  ]
);
<
aviationDB>

```

Figure 17. 2. MongoDB Compass Shell showing denormalization aggregation pipeline (Stage 2–3) [2]

b. The output of the script

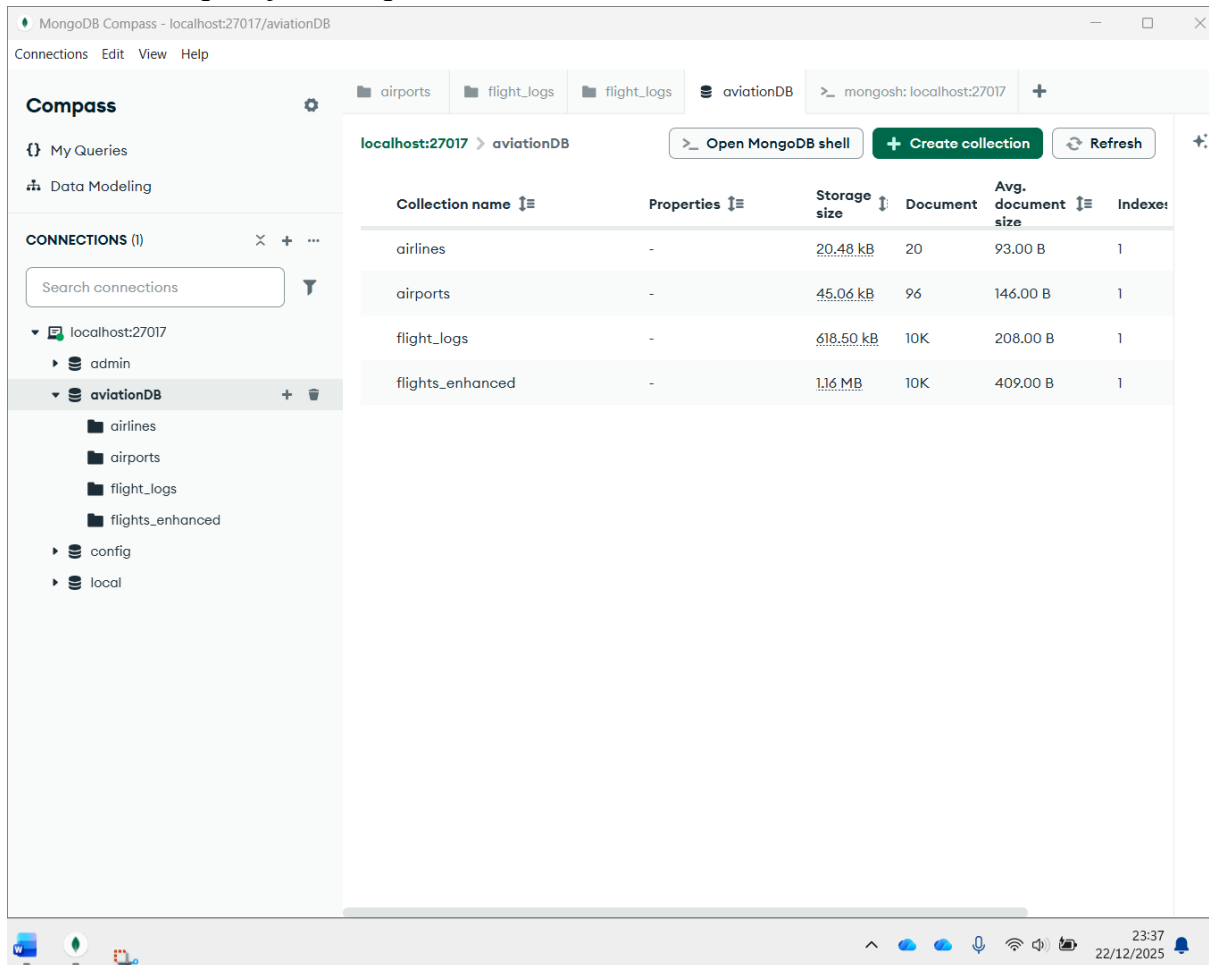


Figure 18. MongoDB Compass showing the output of the aggregation script with the newly created *flights_enhanced* collection in the *aviationDB* database [After with 4 collections]

c. The new denormalized collection in MongoDB Compass

MongoDB Compass - localhost:27017/aviationDB.flights_enhanced

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- admin
- aviationDB
 - airlines
 - airports
 - flight_logs
 - flights_enhanced
- config
- local

localhost:27017 > aviationDB > flights_enhanced

Documents 10K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 25 of 10000

	_id ObjectId	flight_id String	airline_iata String	origin_iata String	dest_iata String
1	ObjectId('69473df08407b63...')	"FL1000"	"CZ"	"PHL"	"DOH"
2	ObjectId('69473df08407b63...')	"FL1001"	"DL"	"DUS"	"RUH"
3	ObjectId('69473df08407b63...')	"FL1002"	"AF"	"DOH"	"SFO"
4	ObjectId('69473df08407b63...')	"FL1003"	"CX"	"SYD"	"ZRH"
5	ObjectId('69473df08407b63...')	"FL1004"	"EK"	"MSP"	"NRT"
6	ObjectId('69473df08407b63...')	"FL1005"	"CX"	"GRU"	"SLC"
7	ObjectId('69473df08407b63...')	"FL1006"	"SU"	"GVA"	"IAH"
8	ObjectId('69473df08407b63...')	"FL1007"	"NH"	"JFK"	"ORD"
9	ObjectId('69473df08407b63...')	"FL1008"	"QR"	"ORD"	"BKK"
10	ObjectId('69473df08407b63...')	"FL1009"	"WN"	"DUB"	"PHX"
11	ObjectId('69473df08407b63...')	"FL1010"	"AF"	"CDG"	"AKL"
12	ObjectId('69473df08407b63...')	"FL1011"	"AA"	"VIE"	"AKL"
13	ObjectId('69473df08407b63...')	"FL1012"	"WN"	"PHL"	"DME"
14	ObjectId('69473df08407b63...')	"FL1013"	"AC"	"WAW"	"CAN"
15	ObjectId('69473df08407b63...')	"FL1014"	"EK"	"PEK"	"JNB"
16	ObjectId('69473df08407b63...')	"FL1015"	"EK"	"DEN"	"GVA"

Figure 19. 1. MongoDB Compass showing the denormalized flights_enhanced collection

MongoDB Compass - localhost:27017/aviationDB.flights_enhanced

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- admin
- aviationDB
 - airlines
 - airports
 - flight_logs
 - flights_enhanced
- config
- local

localhost:27017 > aviationDB > flights_enhanced

Documents 10K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 25 of 10000

	dest_iata String	departure_time Date	arrival_time Date	passenger_count Int32	aircraft_model String
1	"DOH"	2025-09-12T09:21:05.000+0...	2025-09-13T00:19:59.229+0...	255	"Boeing 737 MAX 8"
2	"RUH"	2025-09-11T16:45:32.000+0...	2025-09-11T23:39:37.687+0...	171	"Boeing 767-300ER"
3	"SFO"	2025-09-29T05:20:54.000+0...	2025-09-29T23:01:24.198+0...	353	"Boeing 777-300ER"
4	"ZRH"	2025-09-12T14:23:41.000+0...	2025-09-13T12:34:18.050+0...	379	"Boeing 747-400"
5	"NRT"	2025-09-10T00:28:21.000+0...	2025-09-10T13:06:43.291+0...	192	"Boeing 757-200"
6	"SLC"	2025-09-08T22:00:58.000+0...	2025-09-09T11:42:03.982+0...	289	"ATR 72-600"
7	"IAH"	2025-09-15T02:40:35.000+0...	2025-09-15T13:57:17.585+0...	355	"Airbus A350-900"
8	"ORD"	2025-09-02T02:02:09.000+0...	2025-09-02T04:46:15.238+0...	429	"Airbus A350-900"
9	"BKK"	2025-09-26T23:22:13.000+0...	2025-09-27T17:31:23.011+0...	443	"Airbus A321neo"
10	"PHX"	2025-09-12T12:44:08.000+0...	2025-09-12T23:53:21.273+0...	414	"Airbus A320neo"
11	"AKL"	2025-09-18T12:55:00.000+0...	2025-09-19T12:41:34.250+0...	89	"Airbus A350-900"
12	"AKL"	2025-09-12T17:33:05.000+0...	2025-09-13T16:27:25.966+0...	294	"Airbus A340-600"
13	"DME"	2025-09-05T21:47:50.000+0...	2025-09-06T08:11:14.139+0...	168	"Boeing 757-200"
14	"CAN"	2025-09-09T12:05:59.000+0...	2025-09-09T23:20:37.605+0...	142	"Boeing 747-400"
15	"JNB"	2025-09-02T14:32:38.000+0...	2025-09-03T06:05:43.693+0...	435	"Bombardier Q400"
16	"GVA"	2025-09-01T22:13:51.000+0...	2025-09-02T09:03:34.385+0...	327	"Embraer E190"

Figure 19. 2. MongoDB Compass showing the denormalized flights_enhanced collection

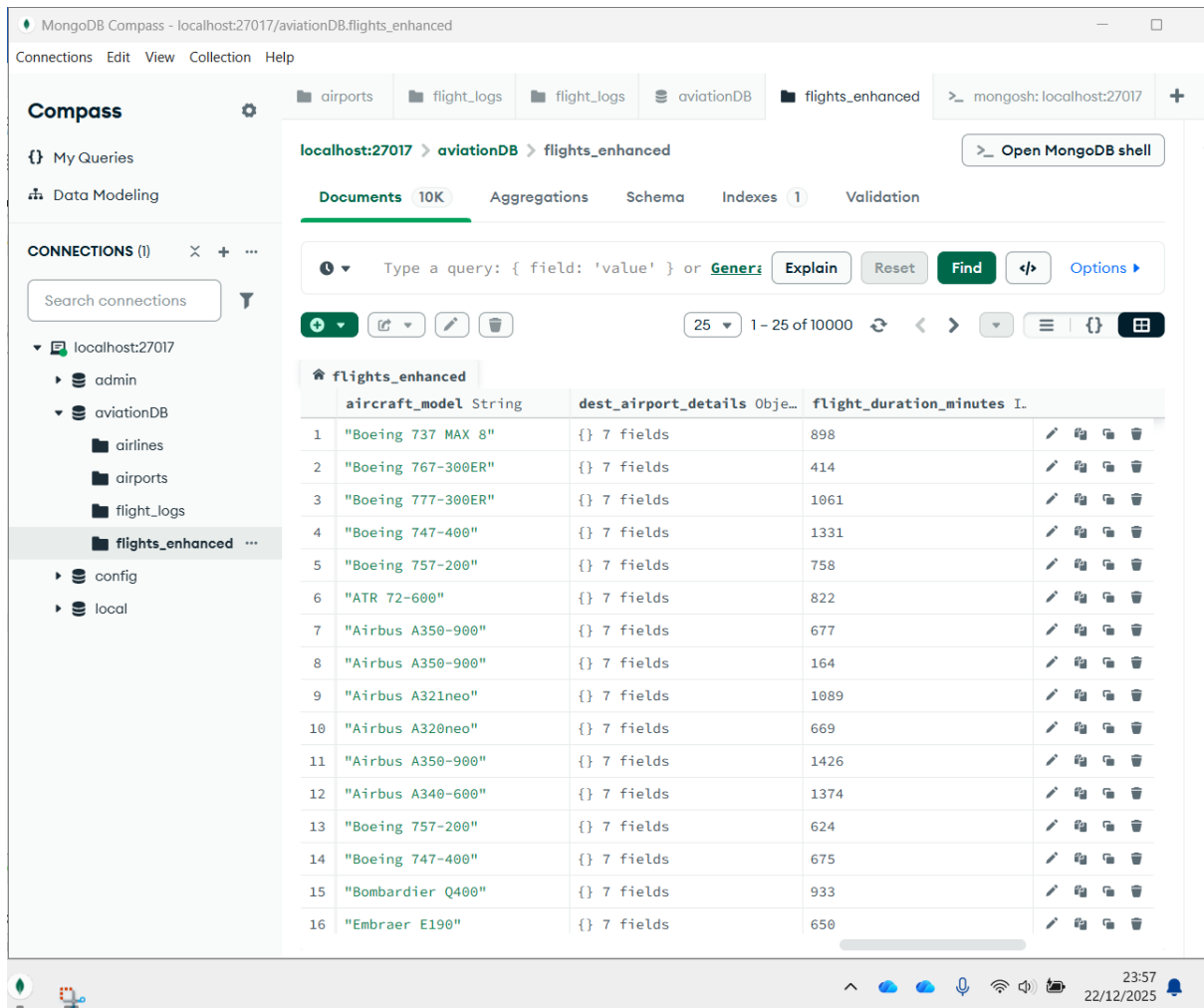


Figure 19. 3. MongoDB Compass showing the denormalized flights_enhanced collection with embedded airport details and computed flight duration

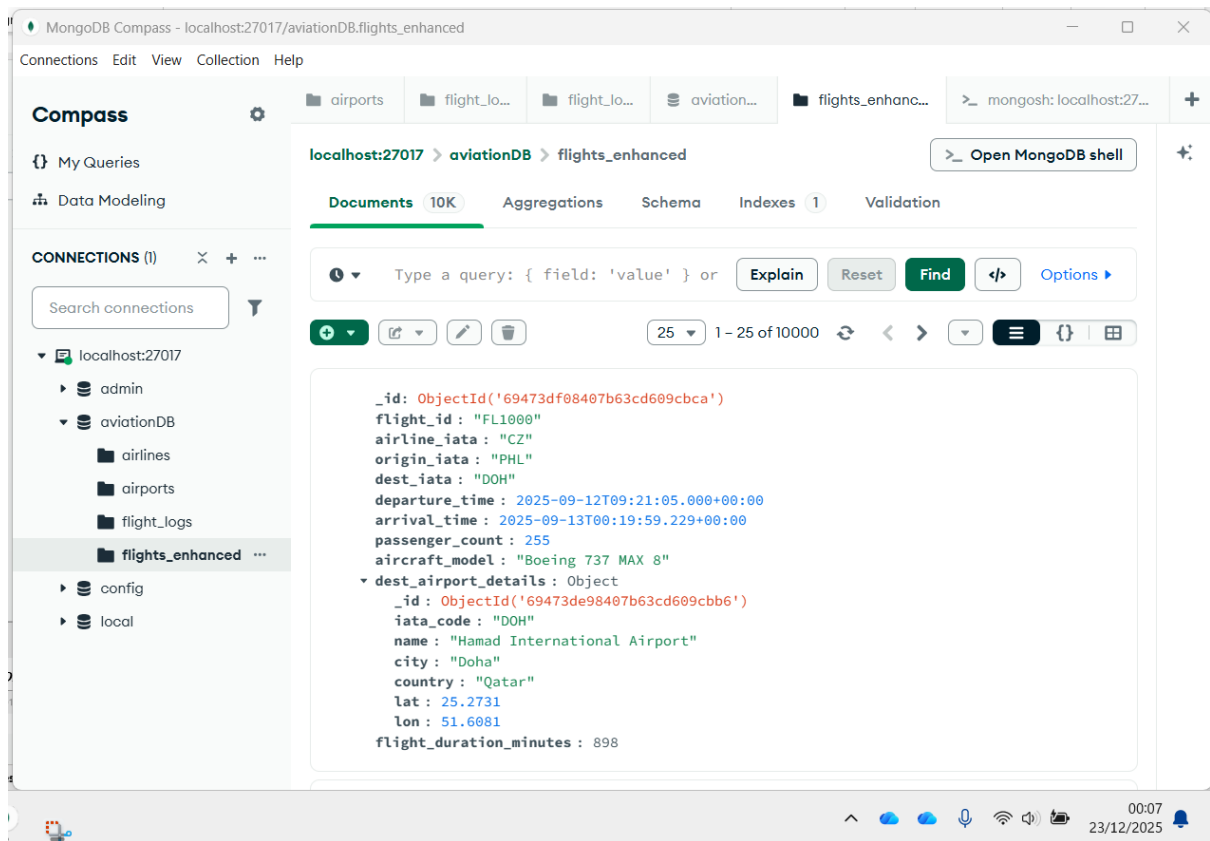


Figure 20. MongoDB Compass showing a sample document from the denormalized `flights_enhanced` collection with embedded `dest_airport_details` and computed `flight_duration_minutes`

Task 5: Advanced Queries

Task 5.1

1. Must provide:

a. Queries title: European Destination Analysis

b. Purpose (Semantic explanation):

The purpose of this query is to track passenger flow into major European destinations by calculating the average number of passengers per flight for cities in France and Germany; in this case, we are trying to observe the top 4 cities with the highest average numbers of passengers.

c. Process (Technical explanation):

First, the \$match filter selects only documents that include flights whose destination country is either France or Germany.

Next, the \$group groups the filtered flights by destination city and calculates the average passenger count for each city using the \$avg operator.

Then, the \$sort orders the results by average passenger count in descending order, so cities with higher passenger flow appear first.

Finally, the \$limit restricts the output to the top four destination cities.

d. The result of the query:

The query returns the top four destination cities in France and Germany with the highest average passenger count per flight.

2. Screenshots of:

a. *The query itself*

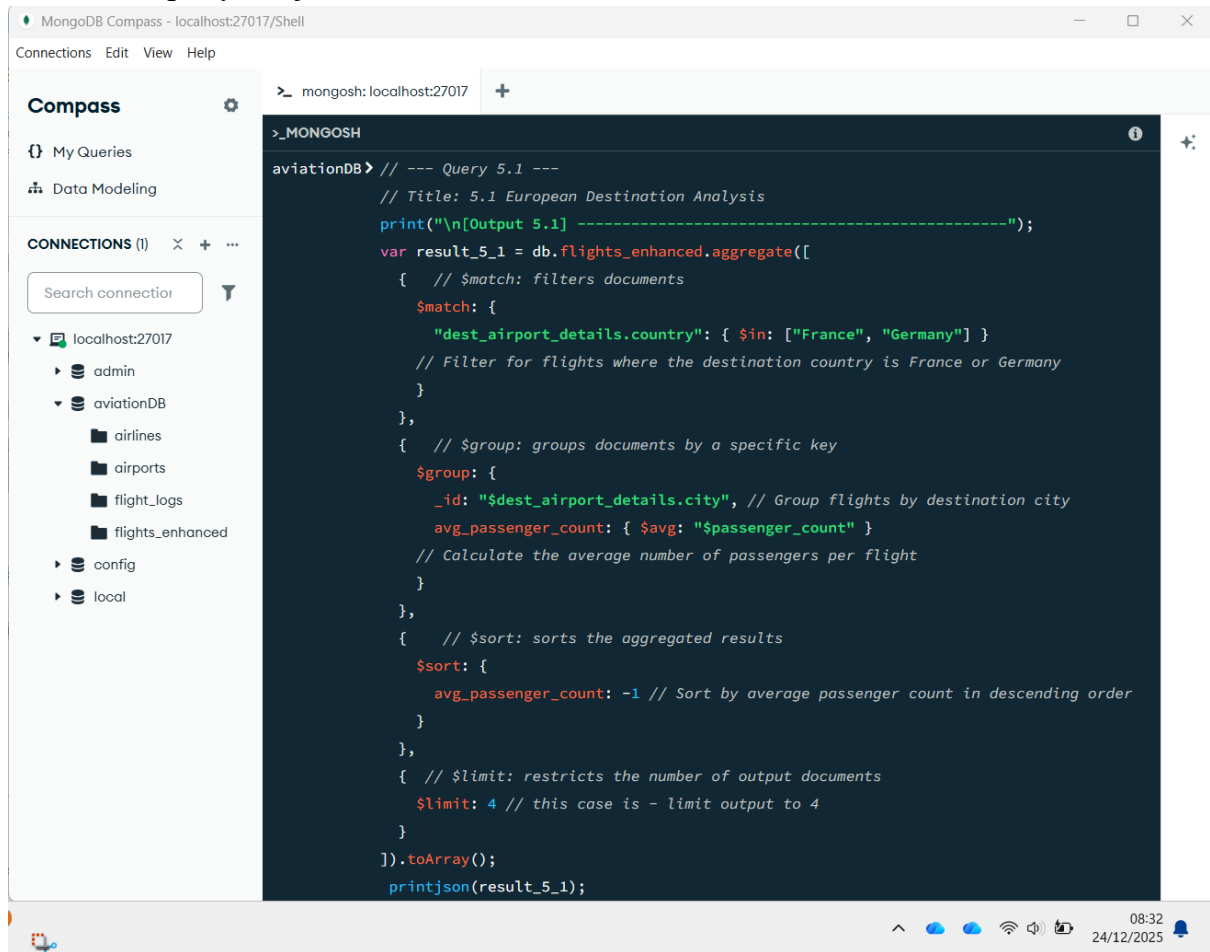


Figure 21. MongoDB Compass mongosh showing the aggregation query for European Destination Analysis

b. *The output of the query*

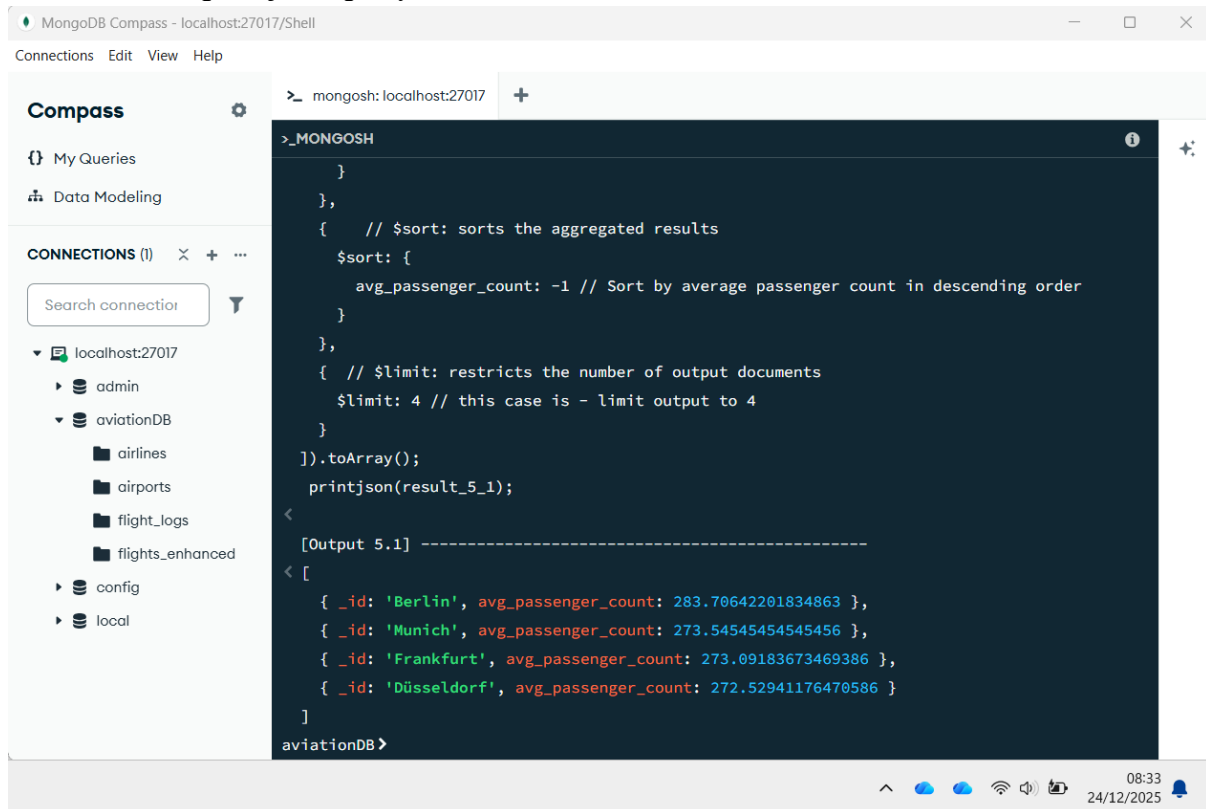


Figure 22. Aggregation query results for European Destination Analysis executed in MongoDB Compass

Task 5.2

1. **Must provide:**

a. **Queries title:** Under-Utilized Long Hauls

b. **Purpose (Semantic explanation):**

The purpose of this query is to identify inefficient long-distance flights' routes by finding routes with long flight durations but a low number of passengers usage.

c. **Process (Technical explanation):**

First, the \$match filters flights that having flight_duration_minutes more than 400 minutes and passenger_count lower than 100 passengers with the \$lt.

Next, the \$project selects only the necessary output fields such as flight_id, destination_country (dest_airport_details.country), passenger_count

Then, the \$sort orders the results by passenger_count in ascending order.

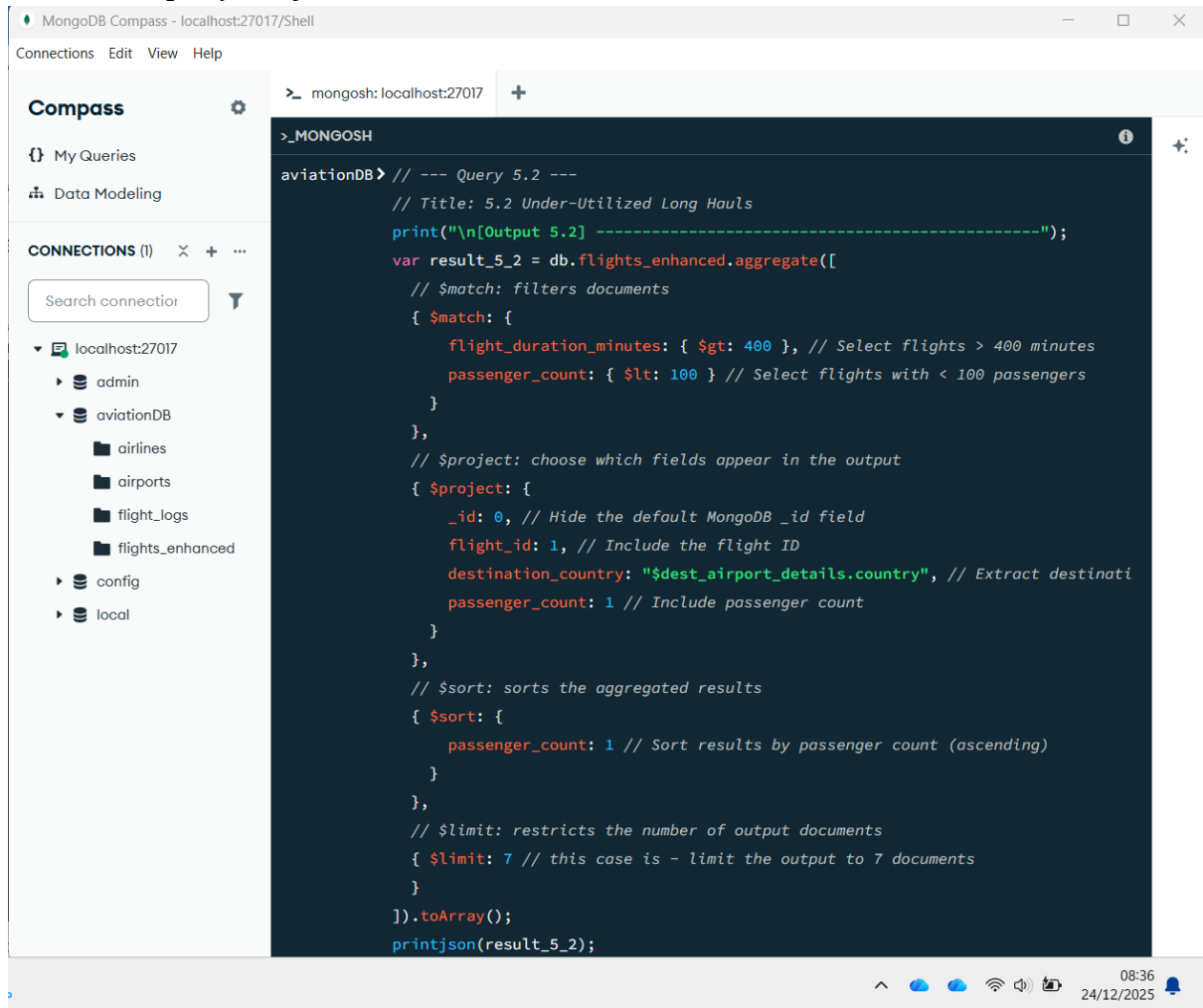
Finally, the \$limit restricts the output to 7 flights.

d. **The result of the query:**

The query returns the 7 longest flights that carry fewer than 100 passengers, ordered from lowest to highest passenger count

2. **Screenshots of:**

a. *The query itself*



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the local connection 'localhost:27017' and the database 'aviationDB'. The main area displays a mongosh terminal window with the following aggregation query:

```
>_ MONGOSH
aviationDB> // --- Query 5.2 ---
// Title: 5.2 Under-Utilized Long Hauls
print("\n[Output 5.2] -----");
var result_5_2 = db.flights_enhanced.aggregate([
  // $match: filters documents
  { $match: {
    flight_duration_minutes: { $gt: 400 }, // Select flights > 400 minutes
    passenger_count: { $lt: 100 } // Select flights with < 100 passengers
  } },
  // $project: choose which fields appear in the output
  { $project: {
    _id: 0, // Hide the default MongoDB _id field
    flight_id: 1, // Include the flight ID
    destination_country: "$dest_airport_details.country", // Extract destination
    passenger_count: 1 // Include passenger count
  } },
  // $sort: sorts the aggregated results
  { $sort: {
    passenger_count: 1 // Sort results by passenger count (ascending)
  } },
  // $limit: restricts the number of output documents
  { $limit: 7 // this case is - limit the output to 7 documents
  }
]).toArray();
printjson(result_5_2);
```

Figure 23. MongoDB Compass mongosh showing the aggregation query for Under-Utilized Long Hauls

b. *The output of the query*

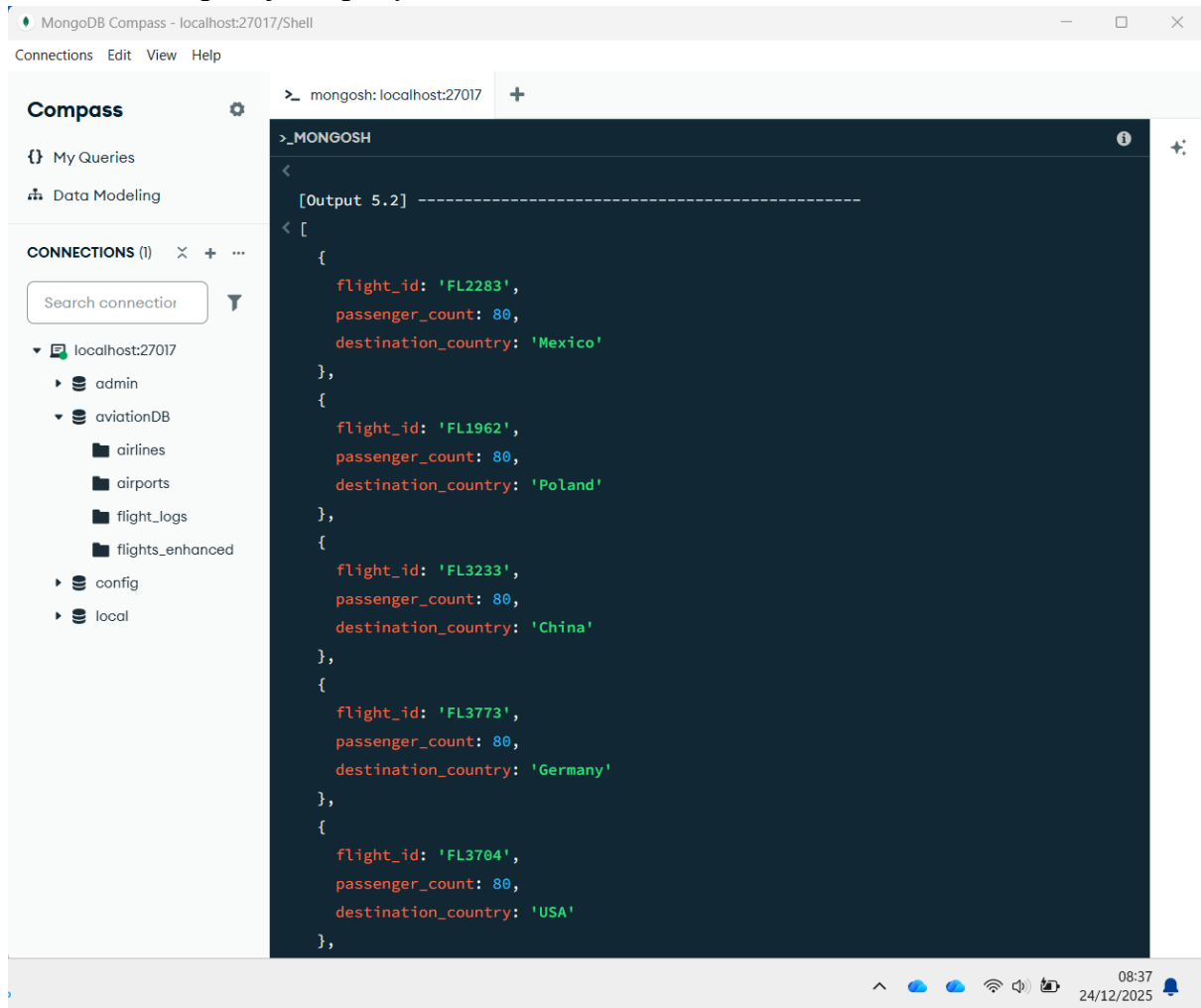


Figure 24. Aggregation query – “First 5 outputs’ results” for Under-Utilized Long Hauls executed in MongoDB Compass

Task 5.3

1. **Must provide:**

a. **Queries title:** Aircraft Range Tier

b. **Purpose (Semantic explanation):** The purpose of this query is to analyse aircraft performance by calculating the average operational range – average flight duration for each aircraft model and identifying models that operate within a medium operational range (in this case is – less than or equal to 650 minutes).

c. **Process (Technical explanation):**

First, the \$group groups all flight records by aircraft_model and computes the average value of flight_duration_minutes for each model.

Next, the \$match filters the grouped results to keep only aircraft models whose average flight duration is less than or equal to 650 minutes with the \$lte.

Then, \$sort orders the filtered aircraft models in descending order based on their average flight duration.

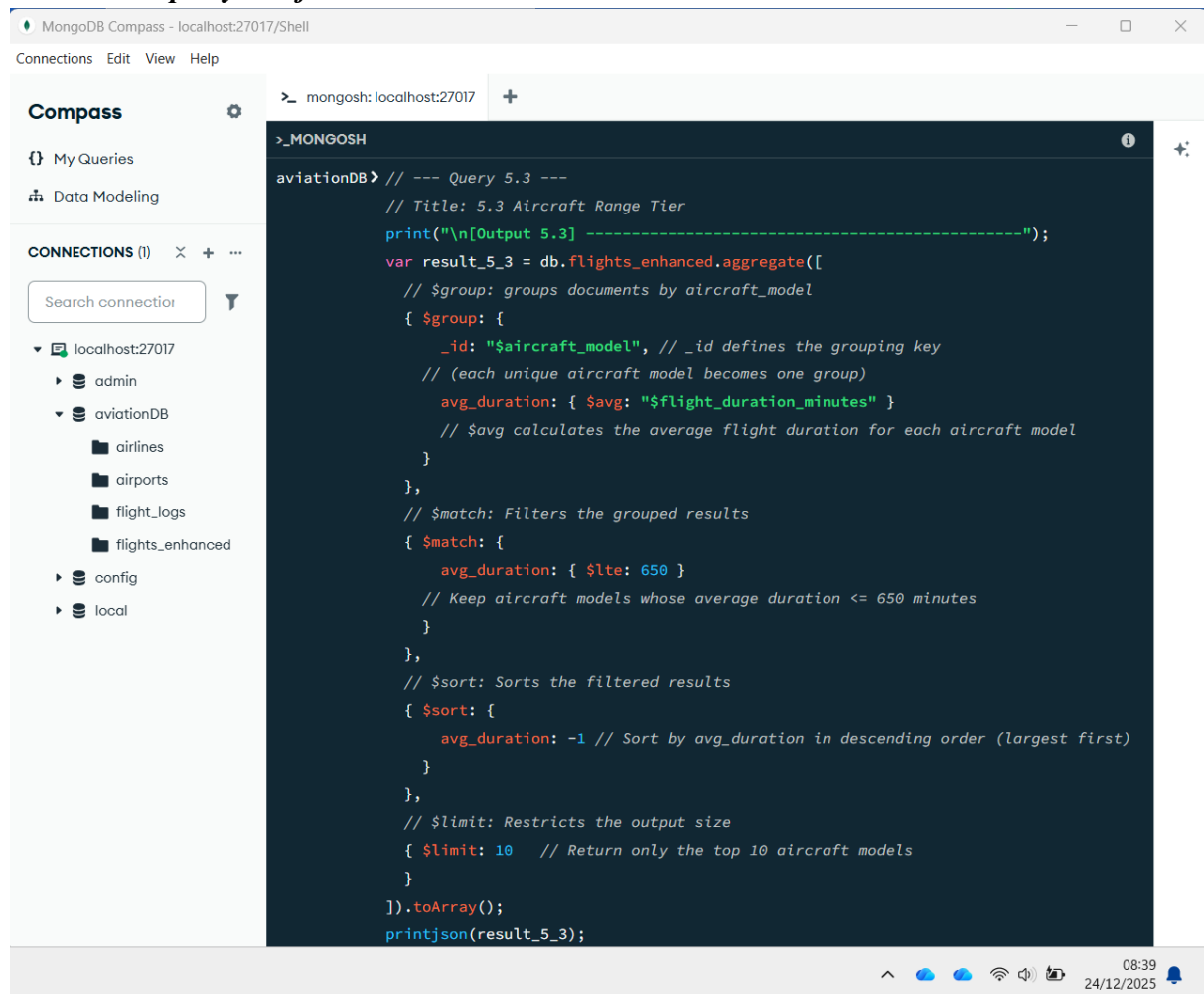
Finally, the \$limit restricts the output to the top 10 aircraft models.

d. The result of the query:

The query returns up to 10 aircraft models whose average operational flight duration does not exceed 650 minutes, sorted from highest to lowest average duration. However, in the actual dataset, only three aircraft models satisfy this condition (average operational flight duration does not exceed 650 minutes), so the query output displays only three results.

2. Screenshots of:

a. The query itself



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the local connection 'localhost:27017' with databases 'admin', 'aviationDB', 'config', and 'local'. The 'aviationDB' database is expanded, showing collections 'airlines', 'airports', 'flight_logs', and 'flights_enhanced'. The main panel displays a mongosh terminal window with the following aggregation query:

```
>_MONGOSH
aviationDB> // --- Query 5.3 ---
// Title: 5.3 Aircraft Range Tier
print("\n[Output 5.3] -----");
var result_5_3 = db.flights_enhanced.aggregate([
  // $group: groups documents by aircraft_model
  { $group: {
    _id: "$aircraft_model", // _id defines the grouping key
    // (each unique aircraft model becomes one group)
    avg_duration: { $avg: "$flight_duration_minutes" }
    // $avg calculates the average flight duration for each aircraft model
  } },
  // $match: Filters the grouped results
  { $match: {
    avg_duration: { $lte: 650 }
    // Keep aircraft models whose average duration <= 650 minutes
  } },
  // $sort: Sorts the filtered results
  { $sort: {
    avg_duration: -1 // Sort by avg_duration in descending order (largest first)
  } },
  // $limit: Restricts the output size
  { $limit: 10 // Return only the top 10 aircraft models
  }
]).toArray();
printjson(result_5_3);
```

Figure 25. MongoDB Compass mongosh showing the aggregation query for Aircraft Range Tier

b. The output of the query

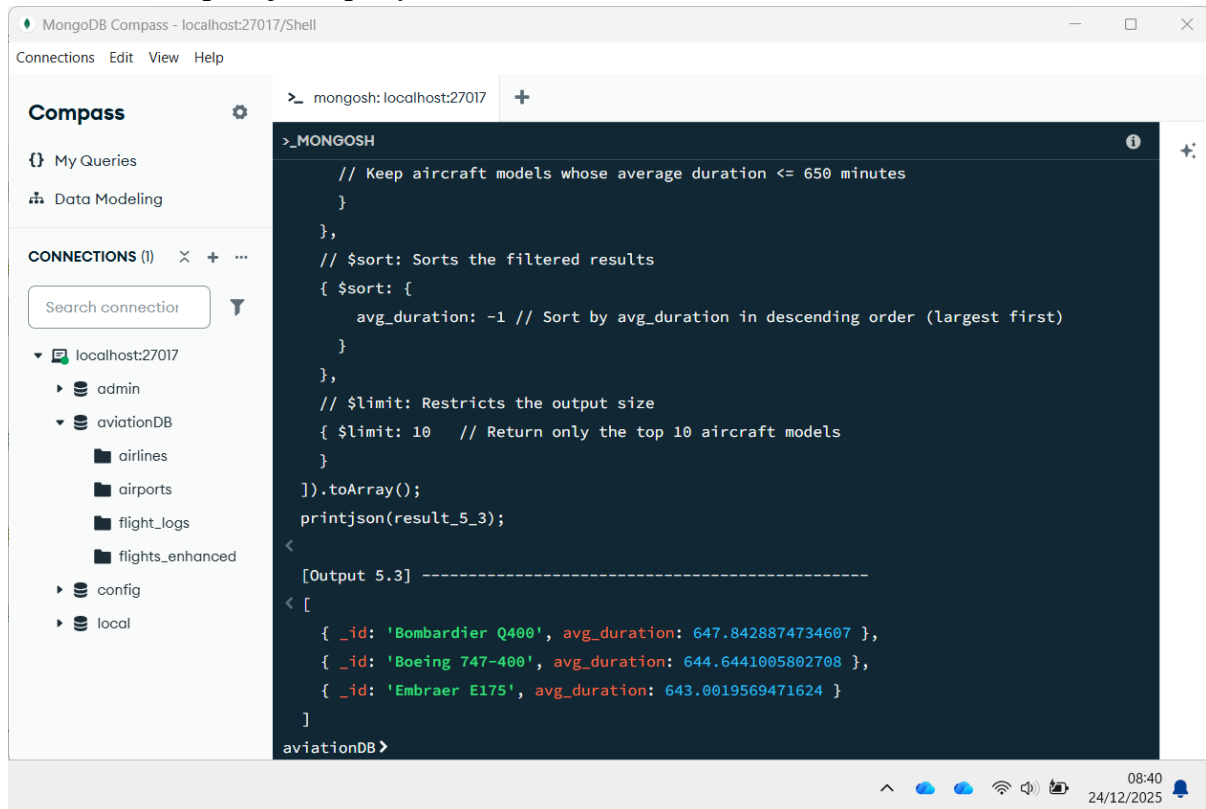


Figure 26. Aggregation query – “Only 3 conditionally satisfied outputs’ results” for Aircraft Range Tier executed in MongoDB Compass

Task 5.4

1. You must provide:

a. Queries title: Busy Route Identification

b. Purpose (Semantic explanation): The purpose of this query is to identify the busy routes by counting how many total flights arrive at each destination airport and selecting the routes with the high frequency.

c. Process (Technical explanation):

First, the \$group groups all flight documents by destination airport code (dest_iata) and calculates the total number of flights (total_flights) for each destination using \$sum.

Next, the \$match filters only those destinations whose total number of flights is greater than or equal to 100 with the \$gte.

Then, the \$sort orders the destinations in ascending order based on total flights.

Finally, the \$limit restricts the final output to the top 5 qualifying routes.

d. The result of the query:

The query returns the five destination airports that have at least 100 incoming flights, ordered from the lowest to the highest flight count among the busy routes.

2. Screenshots of:

a. *The query itself*

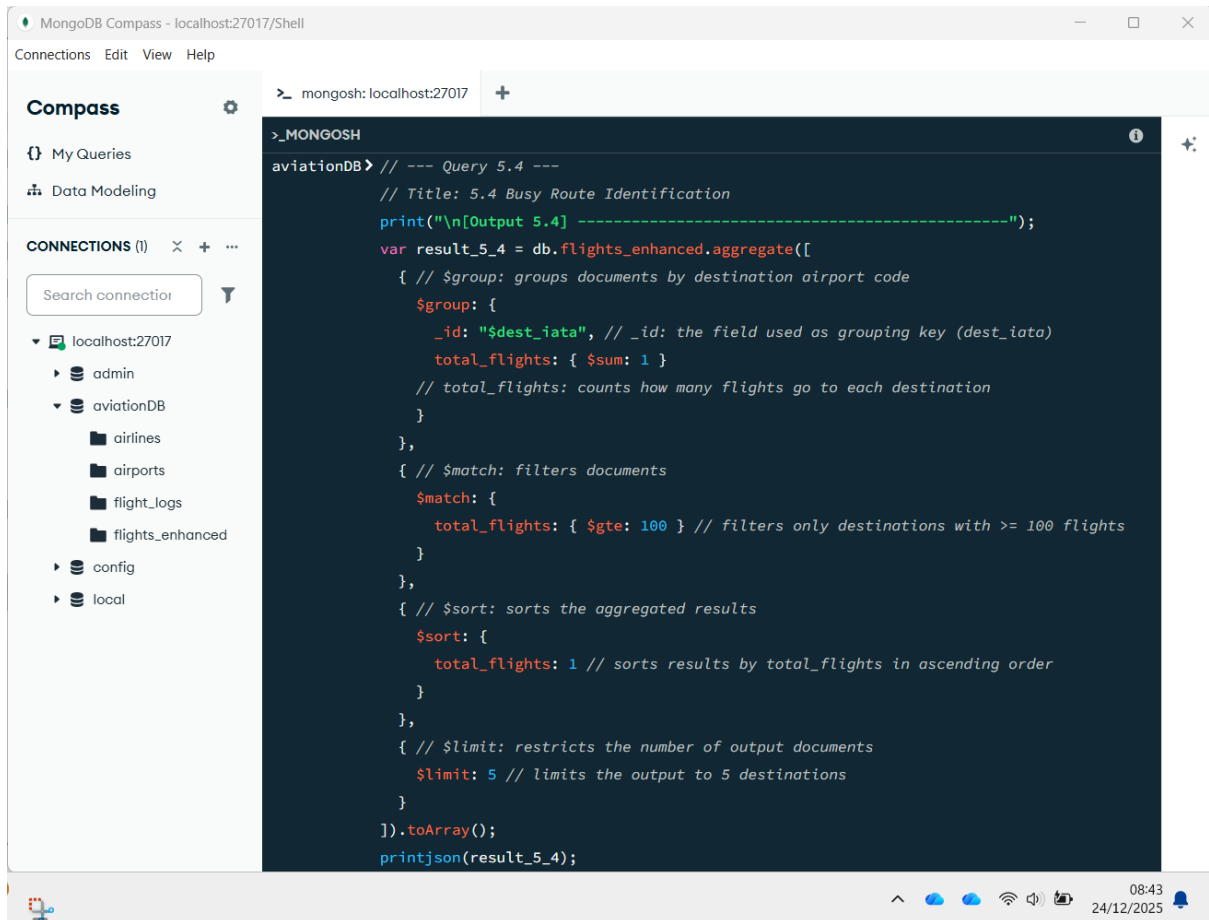


Figure 27. MongoDB Compass mongosh showing the aggregation query for Busy Route Identification

b. The output of the query

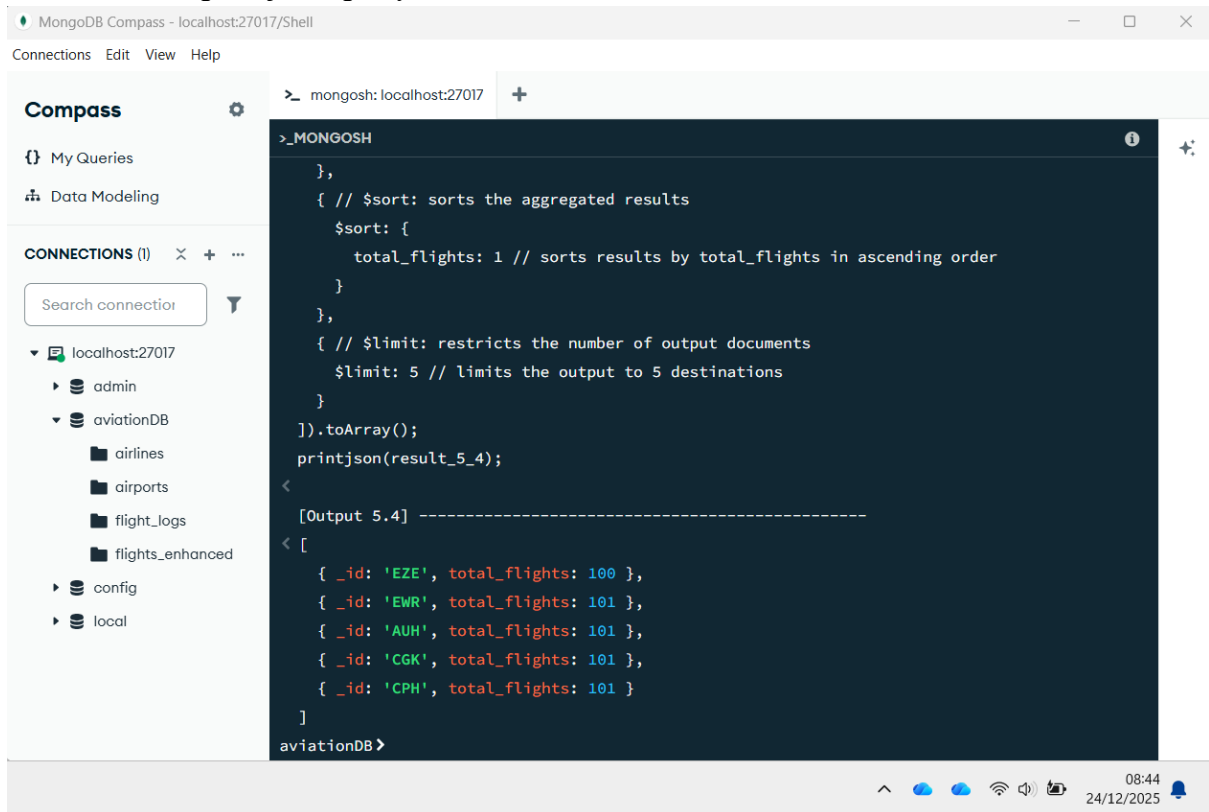


Figure 28. Aggregation query results for Busy Route Identification executed in MongoDB Compass

Conclusions

1. Summary: A summary of the work completed

First of all, I have downloaded/ installed MongoDB (MongoDB Community Server, MongoDB Compass, and MongoDB Shell) for the first task. I also connected to a localhost (localhost:27017) and configured MongoDB Shell. Then, when I open Command Prompt and type mongosh, I am able to interact with the database through the command prompt.

Secondly, for the second task, I learned how to import collections (airlines, airports, flight_logs) in MongoDB Compass.

Thirdly, for the third task, I learned how to change data types (from String to Date, from String to Double, and from String to Int32).

Next, in Task 4, I learned how to denormalize collections by writing and executing a script with an aggregation pipeline to build a new collection called “flights_enhanced” using MongoDB Compass MongoDB Shell.

Finally, in Task 5, I practiced advanced aggregation queries in MongoDB with four specific tasks: European Destination Analysis, Under-Utilized Long Hauls, Aircraft Range Tier, and Busy Route Identification. Through these tasks, I remembered and practiced well and became familiar with the stages \$match, \$group, \$sort, \$limit and the operators such as \$lte (less than or equal to), \$gte (greater than or equal to), \$gt (greater than), \$lt (less than), and also learned more operators such as \$in, \$sum, \$avg.

2. Reflection: Discuss the challenges faced (e.g., working with data types, nesting documents) and what you learned about document databases vs relational databases.

This is an extremely important part of the report

Challenges faced:

The first problem is data conversion, especially converting from String to Date, because the Date data type is complex. Therefore, by using the article from <https://www.geeksforgeeks.org/mongodb/how-to-change-the-type-of-a-field/> and MongoDB tutorials, I applied the \$convert operator and tried to ensure that the data type transformation was performed safely, I also applied the onError and onNull parameters. And the things are mistakes were easy to make and hard to detect without checking the results in MongoDB Compass.

With document structure, MongoDB uses both nested documents and arrays. The \$lookup stage adds a new array field to each input document. \$lookup creates arrays that then require \$unwind. Understanding when to keep arrays versus flatten them was confusing for me at first. For Task 4, denormalization complexity, the pipeline logic was difficult because the order of stages matters, and I had to think clearly in order to structure it correctly.

With Task 4 and the subtasks of Task 5, I failed many times. Sometimes I was missing a comma, and sometimes a stage was missing. Sometimes my fear was that the script was running, but the logic of my script was not correct. Therefore, I often had to check the outputs and rethink whether everything was correct or not.

What you learned about document databases vs relational databases

Document databases are good at managing hierarchical and semi-structured data by using flexible schemas and minimizing joins through embedding.

Instead of eliminating all redundancy as in relational design, document database modeling focuses on how data is accessed, often favoring embedding and denormalization to improve query performance and simplify data retrieval. MongoDB provides schema flexibility, allowing documents in the same collection to have different structures, enabling faster iteration and easier adaptation to changing requirements.

In contrast, relational databases are better suited for strict data integrity, where normalized schemas and joins provide reliability and consistency. They preserve ACID properties and rely heavily on joins to connect related data across tables, which is powerful for structured data. Relational systems enforce strict schemas that require migrations when changes occur.

Regarding joins vs. embedding, while SQL databases rely heavily on joins or normalized tables, embedding is a practice used by MongoDB, where related data is stored within a document. Embedding related information and applying denormalization in Task 4 helped me significantly reduce query complexity, and I could clearly see the performance improvement during the analytical queries in Task 5.

One of the most important technical lessons for me was the benefit of precomputing values such as `flight_duration_minutes`. By calculating it in advance, I reduced the workload during query execution, which made my aggregation pipelines faster, cleaner, and easier to understand.

This project really changed the way I think about database design. Instead of focusing on removing every redundancy like in relational databases, I learned that document databases are more about how the data is actually used. Designing the schema around performance and real access patterns makes much more sense in this context.

3. Answer the theoretical question 1:

a. Provide the question itself: Task 4 asked you to "denormalize" data. In Relational Database design (SQL), this often violates "Third Normal Form" (3NF). Why is this encouraged in MongoDB?

b. Write the answer to the question:

In relational databases (SQL), Third Normal Form (3NF) is encouraged because it minimizes redundancy, prevents update anomalies, and is efficient when performing joins. However, MongoDB is designed with a different priority: it focuses on fast data retrieval using self-contained documents. In many real-world applications, data is read much more often than it is updated, so storing all related information inside a single document allows queries to be faster and simpler, even if it means some controlled duplication of data.

Denormalization is therefore encouraged in MongoDB because it allows applications to fetch all required information in one operation without performing expensive or complex joins. Although MongoDB supports joins using \$lookup, this approach is generally heavier than

reading from an already embedded document, especially for analytical workloads that are executed frequently.

In Task 4, the `flight_logs` collection was originally normalized, but for fast analysis it was necessary to denormalize the data. The new `flights_enhanced` collection was created as an analytics-oriented snapshot by joining airport information once (`$lookup`), flattening the structure (`$unwind`), computing derived values such as flight duration (`$dateDiff`), and storing the final result (`$out`). This design allows the analytical queries in Task 5 to run much faster and with simpler pipelines.

Although denormalization may introduce the risk of data inconsistency if the same airport data is copied into many flight documents, this risk is acceptable in this scenario because the `flights_enhanced` collection is intended to act as a stable snapshot for analysis rather than a live transactional dataset.

Overall, when the data is stored in this denormalized form, the analysis becomes noticeably faster because the system no longer has to perform the same `$lookup` operations over and over during query execution. In practice, analytical tasks usually involve a lot of filtering, grouping, and summarizing of data, so having the airport and airline information already included inside each flight record makes the queries easier to write and much more efficient to run. This design choice also matches well with MongoDB's document-based approach, which prioritizes fast data retrieval instead of strictly following traditional normalization rules.

4. Answer the theoretical question 2:

a. Provide the question itself: Why did the instructions require you to use `printjson()` instead of standard JavaScript `console.log()`?

b. Write the answer to the question:

The reason the instructions ask us to use `printjson()` instead of `console.log()` is simply because MongoDB Shell is not a normal JavaScript environment like the browser or Node.js. It is a special tool built to work with database objects, not regular JavaScript variables.

`printjson()` is made specifically for MongoDB. It knows how to display MongoDB documents in a way that is easy for humans to read. When a document contains nested objects, arrays, `ObjectId`, `Date`, or other BSON data types, `printjson()` formats everything nicely and clearly on the screen. With `console.log()`, these complex objects may appear messy, incomplete, or even confusing inside mongosh.

So in practice, using `printjson()` makes your results easier to understand, easier to verify, and easier to present in screenshots or reports. That is why it is the preferred and recommended way to display query results in MongoDB Shell.